AS7058 Chip Library

API Documentation
revision 4.2.1

Generated by Doxygen

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Acronyms and Abbreviations . . . 2
  - 1.2.1 Acronyms . . . 2
  - 1.2.2 Abbreviations . . . 2

2 ChipLib **2**
- 2.1 Block diagram . . . 2

3 OSAL **3**
- 3.1 Block diagram . . . 3
- 3.2 Function dependencies ChipLib - OSAL . . . 4
- 3.3 Interfaces . . . 5

4 Module Documentation **5**
- 4.1 ChipLib Functions . . . 5
  - 4.1.1 Detailed Description . . . 6
  - 4.1.2 Function Documentation . . . 6
- 4.2 OSAL Functions . . . 13
  - 4.2.1 Detailed Description . . . 13
  - 4.2.2 Typedef Documentation . . . 13
  - 4.2.3 Function Documentation . . . 13
- 4.3 Error Codes . . . 17
  - 4.3.1 Detailed Description . . . 18
  - 4.3.2 Typedef Documentation . . . 18
  - 4.3.3 Enumeration Type Documentation . . . 18
- 4.4 ChipLib Definitions . . . 20
  - 4.4.1 Detailed Description . . . 27
  - 4.4.2 Macro Definition Documentation . . . 27
  - 4.4.3 Typedef Documentation . . . 28
  - 4.4.4 Enumeration Type Documentation . . . 30
- 4.5 Standard Includes . . . 41
  - 4.5.1 Detailed Description . . . 41
  - 4.5.2 Macro Definition Documentation . . . 41
- 4.6 Vital Signs AGC . . . 42
  - 4.6.1 Detailed Description . . . 42
  - 4.6.2 Macro Definition Documentation . . . 42
  - 4.6.3 Typedef Documentation . . . 43
  - 4.6.4 Enumeration Type Documentation . . . 43
- 4.7 Sub-sample extraction . . . 46
  - 4.7.1 Detailed Description . . . 46
  - 4.7.2 Function Documentation . . . 46
- 4.8 Sub-sample extraction with SAR support . . . 48
  - 4.8.1 Detailed Description . . . 48
  - 4.8.2 Macro Definition Documentation . . . 48
  - 4.8.3 Function Documentation . . . 49
  - 4.8.4 Variable Documentation . . . 50

5 Data Structure Documentation **52**
- 5.1 agc_configuration_t Struct Reference . . . 52
  - 5.1.1 Detailed Description . . . 52
  - 5.1.2 Field Documentation . . . 52
- 5.2 agc_status_t Struct Reference . . . 54

5.2.1 Detailed Description . . . 54
5.2.2 Field Documentation . . . 54
5.3 agc_version_t Struct Reference . . . 54
5.3.1 Detailed Description . . . 55
5.3.2 Field Documentation . . . 55
5.4 as7058_bioz_meas_config_t Struct Reference . . . 55
5.4.1 Detailed Description . . . 55
5.4.2 Field Documentation . . . 55
5.5 as7058_bioz_meas_result_t Struct Reference . . . 56
5.5.1 Detailed Description . . . 56
5.5.2 Field Documentation . . . 56
5.6 as7058_bioz_measurement Struct Reference . . . 57
5.6.1 Detailed Description . . . 57
5.6.2 Field Documentation . . . 57
5.7 as7058_eda_scaling_config_t Struct Reference . . . 57
5.7.1 Detailed Description . . . 57
5.7.2 Field Documentation . . . 57
5.8 as7058_eda_scaling_result_t Struct Reference . . . 58
5.8.1 Detailed Description . . . 58
5.8.2 Field Documentation . . . 58
5.9 as7058_extract2_sample_t Struct Reference . . . 59
5.9.1 Detailed Description . . . 59
5.9.2 Field Documentation . . . 59
5.10 as7058_extract2_state_t Struct Reference . . . 60
5.10.1 Detailed Description . . . 60
5.10.2 Field Documentation . . . 60
5.11 as7058_extract_metadata_t Struct Reference . . . 61
5.11.1 Detailed Description . . . 62
5.11.2 Field Documentation . . . 62
5.12 as7058_meas_config_t Struct Reference . . . 62
5.12.1 Detailed Description . . . 62
5.12.2 Field Documentation . . . 63
5.13 as7058_pd_offset_calibration_config_t Struct Reference . . . 63
5.13.1 Detailed Description . . . 64
5.13.2 Field Documentation . . . 64
5.14 as7058_pd_offset_calibration_result_t Struct Reference . . . 64
5.14.1 Detailed Description . . . 64
5.14.2 Field Documentation . . . 65
5.15 as7058_reg_group_control_t Union Reference . . . 65
5.15.1 Detailed Description . . . 65
5.15.2 Field Documentation . . . 65
5.16 as7058_reg_group_ecg_t Union Reference . . . 66
5.16.1 Detailed Description . . . 66
5.16.2 Field Documentation . . . 66
5.17 as7058_reg_group_fifo_t Union Reference . . . 66
5.17.1 Detailed Description . . . 66
5.17.2 Field Documentation . . . 67
5.18 as7058_reg_group_iir_t Union Reference . . . 67
5.18.1 Detailed Description . . . 67
5.18.2 Field Documentation . . . 67
5.19 as7058_reg_group_ios_t Union Reference . . . 68
5.19.1 Detailed Description . . . 68
5.19.2 Field Documentation . . . 68
5.20 as7058_reg_group_led_t Union Reference . . . 68

5.20.1 Detailed Description . . . 68
5.20.2 Field Documentation . . . 69
5.21 as7058_reg_group_pd_t Union Reference . . . 69
5.21.1 Detailed Description . . . 69
5.21.2 Field Documentation . . . 69
5.22 as7058_reg_group_power_t Union Reference . . . 70
5.22.1 Detailed Description . . . 70
5.22.2 Field Documentation . . . 70
5.23 as7058_reg_group_pp_t Union Reference . . . 70
5.23.1 Detailed Description . . . 71
5.23.2 Field Documentation . . . 71
5.24 as7058_reg_group_ppg_t Union Reference . . . 71
5.24.1 Detailed Description . . . 71
5.24.2 Field Documentation . . . 71
5.25 as7058_reg_group_seq_t Union Reference . . . 72
5.25.1 Detailed Description . . . 72
5.25.2 Field Documentation . . . 72
5.26 as7058_reg_group_sinc_t Union Reference . . . 72
5.26.1 Detailed Description . . . 73
5.26.2 Field Documentation . . . 73
5.27 as7058_special_measurement_result_t Union Reference . . . 73
5.27.1 Detailed Description . . . 73
5.27.2 Field Documentation . . . 73
5.28 as7058_reg_group_control_t::control_regs Struct Reference . . . 74
5.28.1 Detailed Description . . . 74
5.28.2 Field Documentation . . . 74
5.29 as7058_reg_group_ecg_t::ecg_regs Struct Reference . . . 75
5.29.1 Detailed Description . . . 75
5.29.2 Field Documentation . . . 75
5.30 extract_metadata Struct Reference . . . 77
5.30.1 Detailed Description . . . 77
5.30.2 Field Documentation . . . 78
5.31 as7058_reg_group_fifo_t::fifo_regs Struct Reference . . . 78
5.31.1 Detailed Description . . . 78
5.31.2 Field Documentation . . . 78
5.32 as7058_reg_group_iir_t::iir_regs Struct Reference . . . 78
5.32.1 Detailed Description . . . 79
5.32.2 Field Documentation . . . 79
5.33 as7058_reg_group_ios_t::ios_regs Struct Reference . . . 79
5.33.1 Detailed Description . . . 80
5.33.2 Field Documentation . . . 80
5.34 as7058_reg_group_led_t::led_regs Struct Reference . . . 82
5.34.1 Detailed Description . . . 82
5.34.2 Field Documentation . . . 83
5.35 as7058_reg_group_pd_t::pd_regs Struct Reference . . . 85
5.35.1 Detailed Description . . . 86
5.35.2 Field Documentation . . . 86
5.36 as7058_reg_group_power_t::power_regs Struct Reference . . . 88
5.36.1 Detailed Description . . . 88
5.36.2 Field Documentation . . . 88
5.37 as7058_reg_group_pp_t::pp_regs Struct Reference . . . 91
5.37.1 Detailed Description . . . 91
5.37.2 Field Documentation . . . 91
5.38 as7058_reg_group_ppg_t::ppg_regs Struct Reference . . . 92

5.38.1 Detailed Description . . . 92
5.38.2 Field Documentation . . . 92
5.39 as7058_reg_group_seq_t::seq_regs Struct Reference . . . 93
5.39.1 Detailed Description . . . 94
5.39.2 Field Documentation . . . 94
5.40 as7058_reg_group_sinc_t::sinc_regs Struct Reference . . . 97
5.40.1 Detailed Description . . . 97
5.40.2 Field Documentation . . . 97
5.41 status_regs Struct Reference . . . 98
5.41.1 Detailed Description . . . 98
5.41.2 Field Documentation . . . 99

1 Introduction

This document provides an overview on how the vital signs sensor AS7058 can be controlled through the ChipLib (Chip Library) and the corresponding OSAL (operating system abstraction layer).

The ChipLib's API allows for usage in several different fields of application. It is programmed in standard C language.

Key features:

- Driver library for convenient configuration and measurement
- Independent of hardware and platform
- Source code tested by ams

Limitations:

- Sensor calibration is not covered by this library.
- The library provides only raw values.

1.1 Overview

The ChipLib can directly be called by the user application and can be used without adaption. Hardware- and platform-dependent aspects must be handled in the OSAL. The OSAL interface is kept simple to keep integration efforts for the customers to a minimum. More information on the API functions can be found in the section OSAL Functions. Details on how to implement application-specific OSAL are provided in the section OSAL.

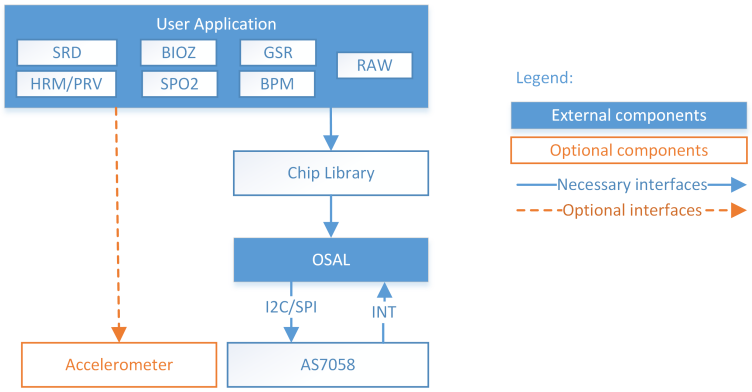

Figure 1 High-Level Block Diagram

- Accelerometer: Separate component providing accelerometer measurements, which are required by the HRM application for best performance.
- AS7058: Physical sensor device.
- OSAL: Hardware- and platform-dependent aspects of ChipLib.
- Chip Library: Control logic for the sensor.
- User Application: Signal processing applications to calculate vital signs data like HRM, SpO2 etc.

1.2 Acronyms and Abbreviations

1.2.1 Acronyms

API = Application Programming Interface
BioZ = Bio Impedance
BPM = Blood Pressure Monitor
EDA = Electrodermal Activity
HRM = Heart Rate Monitor
I2C = Inter-Integrated Circuit
OSAL = Operating System Abstraction Layer
SPI = Serial Peripheral Interface
SpO2 = Oxygen Saturation
SRD = Signal Range Detection

1.2.2 Abbreviations

ChipLib = Chip Library

2 ChipLib

This page describes how to work with the ChipLib in more detail.

2.1 Block diagram

This block diagram shows you all the provided functions of the library. Three groups of functions are available:

- Initialize/shutdown: Starts and stops the ChipLib.
- Configuration: Allows the user to configure the ChipLib.
- Measurement: Handles the measurements.
- Register access: Direct access of register addresses.

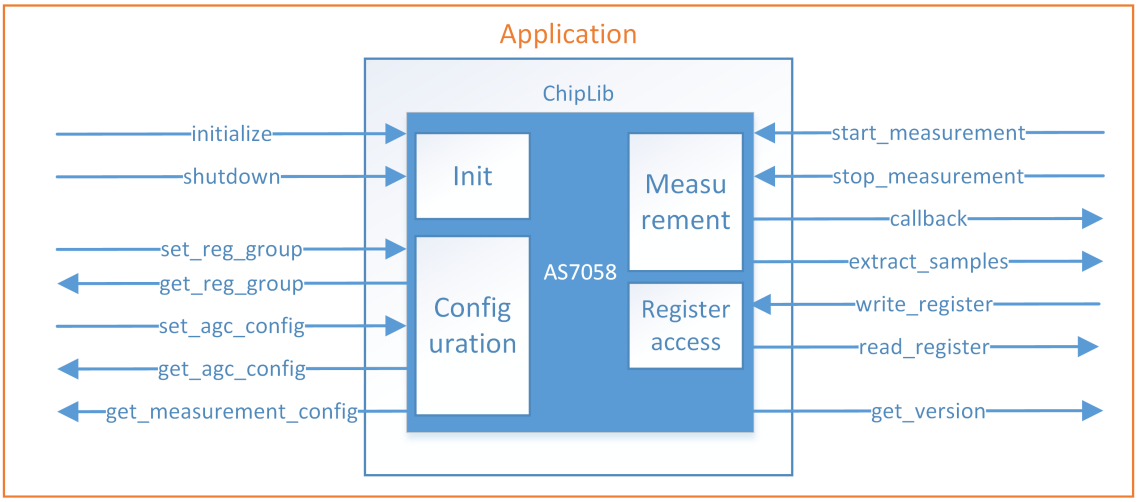

Figure 2 ChipLib API

The functions and their arguments are described in section ChipLib Functions.

3 OSAL

The following pages describe the OSAL (operating system abstraction layer) of the ChipLib. This is used to encapsulate platform- and operating system-specific function to a separate layer, where anybody can customize this interface to their requirements.

This interface is designed to be as simple as possible. No complex data types are used and dependencies between functions are avoided whenever possible. An overview of the OSAL is shown in the next section.

3.1 Block diagram

The following figure describes the dependencies of ChipLib. Inside the OSAL, functionalities such as I2C or SPI and interrupt handling must be implemented.

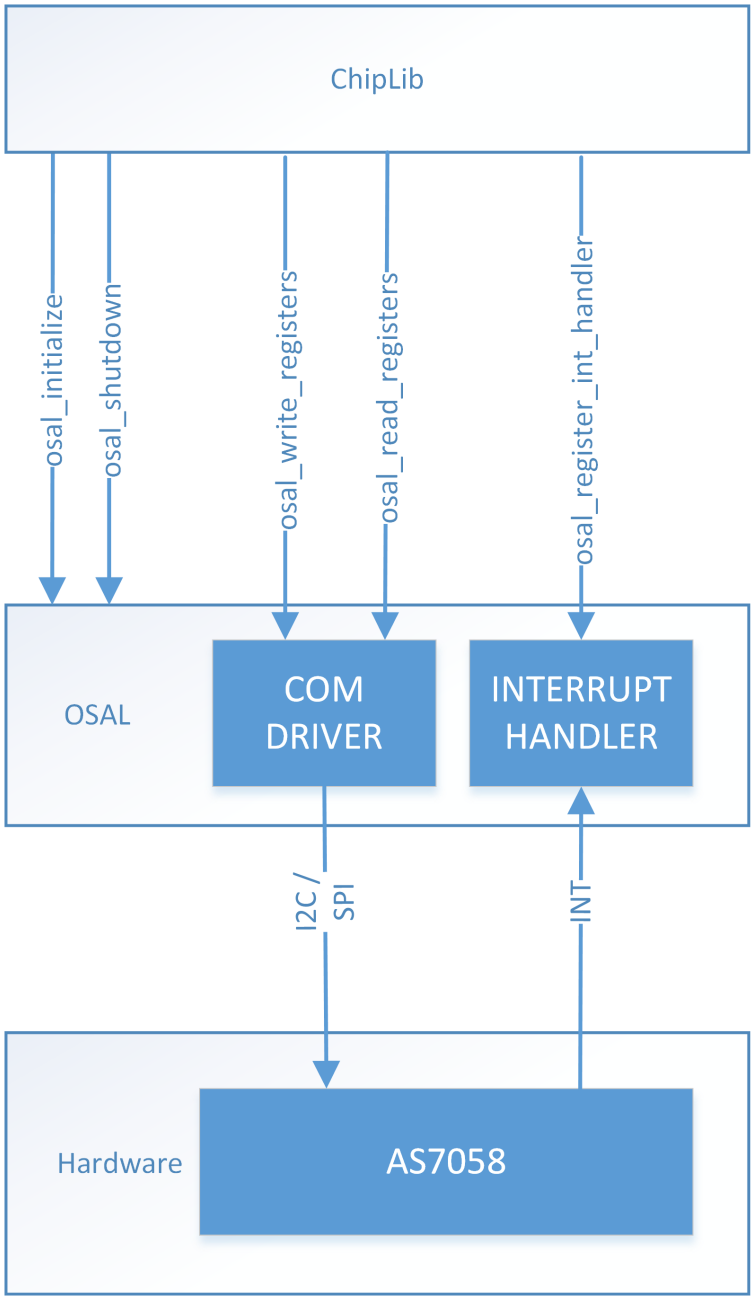

Figure 3 Overview OSAL

Function blocks:

- COM DRIVER: Is used to communicate with the sensor via I2C or SPI.
- INTERRUPT HANDLER: This component handles the interrupt pin and fires an event on falling edge.

3.2 Function dependencies ChipLib - OSAL

The following table describes which OSAL functions are called by which ChipLib functions:

Table 1 Function dependencies ChipLib - OSAL

OSAL functions	ChipLib functions
as7058_osal_initialize	as7058_initialize
as7058_osal_shutdown	as7058_shutdown
as7058_osal_write_registers, as7058_osal_read_registers	as7058_initialize, as7058_shutdown, as7058_write_register, as7058_read_register, as7058_start_measurement, as7058_stop_measurement, as7058_get_measurement_config, as7058_set_reg_group, as7058_get_reg_group, as7058_callback_t
as7058_osal_register_int_handler	as7058_initialize

3.3 Interfaces

The functions and their arguments are described in section OSAL Functions.

4 Module Documentation

4.1 ChipLib Functions

This is the chip library for ams vital signs chip AS7058.

Functions

- err_code_t as7058_initialize (const as7058_callback_t p_normal_callback, const as7058_callback_special_measurement_t p_special_callback, const void *p_cb_param, const char *p_interface_descr)

  Initializes the library and the device.
- err_code_t as7058_shutdown (void)

  Stops all internal actions and power down the device.
- err_code_t as7058_set_reg_group (as7058_reg_group_ids_t id, const uint8_t *p_data, uint8_t size)

  Write a register group.
- err_code_t as7058_get_reg_group (as7058_reg_group_ids_t id, uint8_t *p_data, uint8_t *p_size)

  Read a register group.
- err_code_t as7058_write_register (as7058_reg_addresses_t reg_addr, uint8_t reg_val)

  Write register.
- err_code_t as7058_read_register (as7058_reg_addresses_t reg_addr, uint8_t *p_reg_val)

  Read register.
- err_code_t as7058_get_measurement_config (as7058_meas_config_t *p_meas_config)

  Read the actual measurement configuration, which was set by the register groups.
- err_code_t as7058_set_agc_config (const agc_configuration_t *p_agc_configs, uint8_t agc_config_num)

  Sets the configuration for auto-gain-control (AGC)
- err_code_t as7058_get_agc_config (agc_configuration_t *p_agc_configs, uint8_t *p_agc_config_num)

Gets the configuration for auto-gain-control (AGC)

- err_code_t as7058_start_measurement (as7058_meas_mode_t mode)

  Starts a measurement.

- err_code_t as7058_stop_measurement (void)

  Stops a measurement.

- const char * as7058_get_version (void)

  Requests the version information.

- err_code_t as7058_set_special_measurement_config (as7058_meas_mode_t mode, const void *p_config, uint16_t size)

  Configures a special measurement mode.

4.1.1 Detailed Description

This is the chip library for ams vital signs chip AS7058.

4.1.2 Function Documentation

4.1.2.1 as7058_initialize()

```
err_code_t as7058_initialize (
        const as7058_callback_t p_normal_callback,
        const as7058_callback_special_measurement_t p_special_callback,
        const void * p_cb_param,
        const char * p_interface_descr )
```

Initializes the library and the device.

Following tasks will be done here:

- Initialize hardware abstraction layer.
- Reset chip.

Note

This function must be called at first, otherwise all other functions return with error code.

Parameters

in	*p_normal_callback*	Pointer to normal callback function, see as7058_callback_t.
in	*p_special_callback*	Pointer to special callback function, see as7058_callback_special_measurement_t. Can be NULL if no special measurement is needed.
in	*p_cb_param*	Optional pointer to an application parameter, which will be transmitted with every callback.
in	*p_interface_descr*	Chiplib forwards this interface description to as7058_osal_initialize.

Return values

ERR_SUCCESS	Function returns without error.
ERR_ARGUMENT	An argument is invalid.
ERR_IDENTIFICATION	The specified sensor was not found.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.2 as7058_shutdown() `err_code_t as7058_shutdown ( void )`

Stops all internal actions and power down the device.

Following tasks will be done here:

- Stops measurement, if running.
- Power down the sensor device.
- Shutdown the hardware abstraction layer.
- Block calling of all other functions, but initialize.

Return values

ERR_SUCCESS	Function returns without error.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.3 as7058_set_reg_group() `err_code_t as7058_set_reg_group ( as7058_reg_group_ids_t id, const uint8_t * p_data, uint8_t size )`

Write a register group.

This function configures the sensor by directly writing values to a group of sensor registers.

Parameters

in	*id*	Identification number of the register group, see as7058_reg_group_ids.
in	*p_data*	Pointer to the register data, like as7058_reg_group_led_t.
in	*size*	Sets the size of register data in bytes.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize first.
ERR_ARGUMENT	Register group id is not supported.
ERR_POINTER	Detected NULL pointer for p_data.
ERR_SIZE	Size of the data buffer is wrong.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.4 as7058_get_reg_group()

```
err_code_t as7058_get_reg_group (
        as7058_reg_group_ids_t id,
        uint8_t * p_data,
        uint8_t * p_size )
```

Read a register group.

Reads the actual register data into a register group structure.

Parameters

in	*id*	Identification number of the register group, see as7058_reg_group_ids.
out	*p_data*	Pointer, where the data of the register group can be saved.
in,out	*p_size*	IN: Maximum buffer size, OUT: Size in byte of the register group data The maximum size is defined in AS7058_MAX_GROUP_SIZE.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_ARGUMENT	Register group id is not supported.
ERR_POINTER	Detected NULL pointer on arguments.
ERR_SIZE	Size of the data buffer is wrong.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.5 as7058_write_register()

```
err_code_t as7058_write_register (
        as7058_reg_addresses_t reg_addr,
        uint8_t reg_val )
```

Write register.

This function sets the value of a single sensor register.

Parameters

in	*reg_addr*	Register address of the sensor, see as7058_reg_addresses.
in	*reg_val*	New register value, which will be written to the register address.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.6 as7058_read_register()

```
err_code_t as7058_read_register (
        as7058_reg_addresses_t reg_addr,
        uint8_t * p_reg_val )
```

Read register.

This function gets the value of a single sensor register.

Parameters

in	*reg_addr*	Register address of the sensor, see as7058_reg_addresses.
out	*p_reg_val*	Actual register value, which was read from the register address.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_POINTER	Detected NULL pointer for p_reg_val.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.7 as7058_get_measurement_config()

```
err_code_t as7058_get_measurement_config (
        as7058_meas_config_t * p_meas_config )
```

Read the actual measurement configuration, which was set by the register groups.

Note

The measurement configuration can be changed after set of new register blocks or writting to single registers. Therefore, read it back after finished configuration to get the actual values.

Parameters

out	*p_meas_config*	Actual measurement configuration, See as7058_meas_config_t

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_POINTER	Detected NULL pointer for p_meas_config.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.8 as7058_set_agc_config()

```
err_code_t as7058_set_agc_config (
    const agc_configuration_t * p_agc_configs,
    uint8_t agc_config_num )
```

Sets the configuration for auto-gain-control (AGC)

Parameters

in	*p_agc_configs*	Pointer to store AGC configuration information for up to 4 channels.
in	*agc_config_num*	Number of given channels.

Return values

ERR_SUCCESS	Function returns without error.
ERR_ARGUMENT	At least one parameter is wrong.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_POINTER	Detected NULL pointer for p_agc_config.

4.1.2.9 as7058_get_agc_config()

```
err_code_t as7058_get_agc_config (
    agc_configuration_t * p_agc_configs,
    uint8_t * p_agc_config_num )
```

Gets the configuration for auto-gain-control (AGC)

Parameters

out	*p_agc_configs*	Pointer to store AGC configuration for activated channels.
in,out	*p_agc_config_num*	Number of channels. IN: Defines how how many AGC channel configurations can be saved inside p_agc_configs. OUT: number of enabled AGC channels.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_POINTER	Detected NULL pointer for p_agc_config.

4.1.2.10 as7058_start_measurement() `err_code_t as7058_start_measurement ( as7058_meas_mode_t mode )`

Starts a measurement.

Parameters

in	*mode*	Measurement mode, see as7058_meas_mode.

Return values

ERR_SUCCESS	Function returns without error.
ERR_ARGUMENT	Invalid measurement mode provided.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_DATA_TRANSFER	Error during communication with sensor.
ERR_CONFIG	No sub-sample is enabled, AOC and AGC are enabled in parallel, or sensor is not properly configured for impedance scaling.

4.1.2.11 as7058_stop_measurement() `err_code_t as7058_stop_measurement ( void )`

Stops a measurement.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to the library is blocked, call as7058_initialize at first.
ERR_DATA_TRANSFER	Error during communication with sensor.

4.1.2.12 as7058_get_version() `const char* as7058_get_version ( void )`

Requests the version information.

Returns

Version string.

4.1.2.13 as7058_set_special_measurement_config()

```
err_code_t as7058_set_special_measurement_config (
            as7058_meas_mode_t mode,
            const void * p_config,
            uint16_t size )
```

Configures a special measurement mode.

Before starting a measurement of mode AS7058_MEAS_MODE_SPECIAL_BIOZ or AS7058_MEAS_MODE_SPECIAL_SCALING_E the corresponding measurement mode needs to be configured.

Parameters

in	*mode*	Measurement mode. See as7058_meas_mode.
in	*p_config*	Pointer to the configuration structure for the special measurement mode. See as7058_eda_scaling_config_t and as7058_bioz_meas_config_t.
in	*size*	Size of the configuration structure.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Access to this function is blocked. Depending on the current state of the library, call as7058_initialize or as7058_stop_measurement.
ERR_ARGUMENT	Provided configuration contains an invalid value.
ERR_POINTER	Detected NULL pointer for p_config.
ERR_SIZE	The size of the configuration does not fit to the mode.
ERR_CONFIG	Special measurement callback was not set when as7058_initialize was called.

4.2 OSAL Functions

This is the abstraction layer for the chip library.

Typedefs

- typedef err_code_t(∗ as7058_osal_interrupt_t) (void)
  Callback function which will be called if a new interrupt notification is available.

Functions

- err_code_t as7058_osal_initialize (const char ∗p_interface_desc)
  Initialization of the hardware abstraction layer.
- err_code_t as7058_osal_shutdown (void)
  Shutdown of the hardware abstraction layer.
- err_code_t as7058_osal_write_registers (uint8_t address, uint16_t number, const uint8_t ∗p_values)
  Set register values inside the chip.
- err_code_t as7058_osal_read_registers (uint8_t address, uint16_t number, uint8_t ∗p_values)
  Get register values from the chip.
- err_code_t as7058_osal_register_int_handler (const as7058_osal_interrupt_t callback_function)
  Registers and unregisters an interrupt handler.

4.2.1 Detailed Description

This is the abstraction layer for the chip library.

Some functions of the chip library are dependent on the operating system. These functions concern initialization, shutdown, I2C transfers, and interrupt handling. They need to be implemented specifically for each application.

4.2.2 Typedef Documentation

4.2.2.1 as7058_osal_interrupt_t `typedef err_code_t(* as7058_osal_interrupt_t) (void)`

Callback function which will be called if a new interrupt notification is available.

This callback function is registered using the function as7058_osal_register_int_handler.

4.2.3 Function Documentation

4.2.3.1 as7058_osal_initialize() `err_code_t as7058_osal_initialize ( const char * p_interface_desc )`

Initialization of the hardware abstraction layer.

- Initialization of global parameters.
- Opens the interface to the sensor.

Note

This function must be called first!

Parameters

in	*p_interface_desc*	Can be used to transfer special initialization data like interface description.

Return values

ERR_SUCCESS	Function returns without error.
ERR_ARGUMENT	Argument content is not supported.
ERR_COM_INTERFACE	The interface to the sensor is faulty.

4.2.3.2 as7058_osal_shutdown() `err_code_t as7058_osal_shutdown ( void )`

Shutdown of the hardware abstraction layer.

- Closes the interface to the sensor
- Deactivates the ENABLE-Pin

Note

This function must be called for cleanup

Return values

ERR_SUCCESS	Function returns without error.
ERR_COM_INTERFACE	The interface to the sensor is faulty.

4.2.3.3 as7058_osal_write_registers()

```
err_code_t as7058_osal_write_registers (
        uint8_t address,
        uint16_t number,
        const uint8_t * p_values )
```

Set register values inside the chip.

Parameters

in	*address*	Register address of the first register to write.
in	*number*	Number of registers to write. The register address is auto-incremented by the sensor.
in	*p_values*	Pointer to register values.

Return values

ERR_SUCCESS	Function returns without error.
ERR_POINTER	NULL pointer detected.
ERR_DATA_TRANSFER	Data transfer error, like bus error or timeout.
ERR_PERMISSION	Library was not initialized using as7058_osal_initialize.

4.2.3.4 as7058_osal_read_registers()

```
err_code_t as7058_osal_read_registers (
        uint8_t address,
        uint16_t number,
        uint8_t * p_values )
```

Get register values from the chip.

Parameters

in	*address*	Register address of the first register to read.
in	*number*	Number of registers to read. The register address is auto-incremented by the sensor.
out	*p_values*	Pointer to buffer where the read register values are written to.

Return values

ERR_SUCCESS	Function returns without error.
ERR_POINTER	NULL pointer detected.
ERR_DATA_TRANSFER	Data transfer error, like bus error or timeout.
ERR_PERMISSION	Library was not initialized using as7058_osal_initialize.

4.2.3.5 as7058_osal_register_int_handler()

```
err_code_t as7058_osal_register_int_handler (
        const as7058_osal_interrupt_t callback_function )
```

Registers and unregisters an interrupt handler.

If the function pointer is NULL, then the old callback will be unregistered.

Parameters

in	*callback_function*	Pointer to callback function or NULL.

Return values

ERR_SUCCESS	Function returns without error.
ERR_PERMISSION	Library was not initialized using as7058_osal_initialize.

4.3 Error Codes

Typedefs

- typedef enum error_codes err_code_t

Enumerations

- enum error_codes {
  ERR_SUCCESS = 0,
  ERR_PERMISSION = 1,
  ERR_MESSAGE = 2,
  ERR_MESSAGE_SIZE = 3,
  ERR_POINTER = 4,
  ERR_ACCESS = 5,
  ERR_ARGUMENT = 6,
  ERR_SIZE = 7,
  ERR_NOT_SUPPORTED = 8,
  ERR_TIMEOUT = 9,
  ERR_CHECKSUM = 10,
  ERR_OVERFLOW = 11,
  ERR_EVENT = 12,
  ERR_INTERRUPT = 13,
  ERR_TIMER_ACCESS = 14,
  ERR_LED_ACCESS = 15,
  ERR_TEMP_SENSOR_ACCESS = 16,
  ERR_DATA_TRANSFER = 17,
  ERR_FIFO = 18,
  ERR_OVER_TEMP = 19,
  ERR_IDENTIFICATION = 20,
  ERR_COM_INTERFACE = 21,
  ERR_SYNCHRONISATION = 22,
  ERR_PROTOCOL = 23,
  ERR_MEMORY = 24,
  ERR_THREAD = 25,
  ERR_SPI = 26,
  ERR_DAC_ACCESS = 27,
  ERR_I2C = 28,
  ERR_NO_DATA = 29,
  ERR_SYSTEM_CONFIG = 30,
  ERR_USB_ACCESS = 31,
  ERR_ADC_ACCESS = 32,
  ERR_SENSOR_CONFIG = 33,
  ERR_SATURATION = 34,
  ERR_MUTEX = 35,
  ERR_ACCELEROMETER = 36,
  ERR_CONFIG = 37,
  ERR_BLE = 38,
  ERR_FILE = 39,
  ERR_DATA = 40,
  ERR_BUSY = 41 }

4.3.1 Detailed Description

Generic error codes used by ams libraries.

4.3.2 Typedef Documentation

4.3.2.1 err_code_t `typedef enum error_codes err_code_t`

This definition will be used for function return values.

4.3.3 Enumeration Type Documentation

4.3.3.1 error_codes `enum error_codes`

Error Codes.

Enumerator

ERR_SUCCESS	Operation was successful.
ERR_PERMISSION	Operation is not permitted.
ERR_MESSAGE	Message is invalid. (Unsupported message type, incorrect CRC, ...)
ERR_MESSAGE_SIZE	Message has the wrong size.
ERR_POINTER	Pointer is invalid. (NULL pointer, pointer wrong memory region, ...)
ERR_ACCESS	Access is denied.
ERR_ARGUMENT	Argument is invalid.
ERR_SIZE	An argument has the wrong size.
ERR_NOT_SUPPORTED	Function is not supported or not implemented.
ERR_TIMEOUT	Operation timed out.
ERR_CHECKSUM	Checksum comparision failed.
ERR_OVERFLOW	Data overflow occurred.
ERR_EVENT	Getting or setting an event failed. (Event queue is full or empty, unexpected event received, ...)
ERR_INTERRUPT	Getting or setting an interrupt failed. (Interrupt resource is not available, ...)
ERR_TIMER_ACCESS	Accessing the timer peripheral failed.
ERR_LED_ACCESS	Accessing the LED peripheral failed.
ERR_TEMP_SENSOR_ACCESS	Accessing the temperature sensor failed.
ERR_DATA_TRANSFER	Communication error occurred.
ERR_FIFO	FIFO operation failed.
ERR_OVER_TEMP	Overtemperature detected.

Enumerator

ERR_IDENTIFICATION	Sensor identification failed.
ERR_COM_INTERFACE	Generic communication interface error. (Communication interface is not available, error while opening or closing a communication interface, ...)
ERR_SYNCHRONISATION	Synchronization error occurred.
ERR_PROTOCOL	Generic protocol error occurred.
ERR_MEMORY	Memory allocation error occurred.
ERR_THREAD	Thread handling operation failed.
ERR_SPI	Accessing the SPI peripheral failed.
ERR_DAC_ACCESS	Accessing the DAC peripheral failed.
ERR_I2C	Accessing the I2C peripheral failed.
ERR_NO_DATA	No data available.
ERR_SYSTEM_CONFIG	System configuration failed. (System resource is not available, system resource generates an error, ...)
ERR_USB_ACCESS	Accessing the USB peripheral failed.
ERR_ADC_ACCESS	Accessing the ADC peripheral failed.
ERR_SENSOR_CONFIG	Sensor configuration failed.
ERR_SATURATION	Saturation detected.
ERR_MUTEX	Mutex handling operation filed.
ERR_ACCELEROMETER	Accessing the accelerometer failed.
ERR_CONFIG	Software component is unusable due to incomplete or incorrect configuration.
ERR_BLE	BLE stack handling operation failed.
ERR_FILE	File handling operation failed.
ERR_DATA	Internal data inconsistency detected.
ERR_BUSY	Module is busy.

4.4 ChipLib Definitions

Description of the used data types.

Data Structures

- struct status_regs
- union as7058_reg_group_power_t
- union as7058_reg_group_control_t
- union as7058_reg_group_led_t
- union as7058_reg_group_pd_t
- union as7058_reg_group_ios_t
- union as7058_reg_group_ppg_t
- union as7058_reg_group_ecg_t
- union as7058_reg_group_sinc_t
- union as7058_reg_group_iir_t
- union as7058_reg_group_seq_t
- union as7058_reg_group_pp_t
- union as7058_reg_group_fifo_t
- struct as7058_meas_config_t
- struct as7058_eda_scaling_config_t
- struct as7058_bioz_meas_config_t
- struct as7058_eda_scaling_result_t
- struct as7058_bioz_measurement
- struct as7058_bioz_meas_result_t
- struct as7058_pd_offset_calibration_config_t
- struct as7058_pd_offset_calibration_result_t
- union as7058_special_measurement_result_t

Macros

- #define AS7058_SILICON_ID 0x92
- #define AS7058_MAX_GROUP_SIZE sizeof(as7058_reg_group_iir_t)
- #define AS7058_FIFO_SAMPLE_SIZE 3
- #define AS7058_FIFO_DATA_BUFFER_SIZE 1536
- #define AS7058_SAMPLE_MAX_VALUE 0xFFFFF
- #define AS7058_PD_OFFSET_CALIBRATION_COMPENSATION_TABLE_ELEMENTS_NUM 15
- #define M_AS7058_SUB_SAMPLE_ID_TO_FLAG(sub_sample_id) (as7058_sub_sample_flags_t)(1 << ((sub_sample_id)-1))

Typedefs

- typedef uint8_t as7058_reg_group_ids_t
- typedef uint32_t as7058_sub_sample_flags_t
- typedef uint8_t as7058_sub_sample_ids_t
- typedef uint8_t as7058_modulator_t
- typedef uint8_t as7058_modulator_sub_sample_t
- typedef uint8_t as7058_fifo_data_marker_t
- typedef uint8_t as7058_meas_mode_t
- typedef uint8_t as7058_reg_addresses_t
- typedef struct status_regs as7058_status_events_t
- typedef uint8_t as7058_sar_transfer_mode_t
- typedef void(∗ as7058_callback_t) (err_code_t error, const uint8_t ∗p_fifo_data, uint16_t fifo_data_size, const agc_status_t ∗p_agc_statuses, uint8_t agc_statuses_num, as7058_status_events_t sensor_events, const void ∗p_cb_param)

  Callback function used to provide regular measurement results to the application.
- typedef void(∗ as7058_callback_special_measurement_t) (as7058_meas_mode_t mode, const as7058_special_measurement_ ∗p_result, uint16_t result_size, const void ∗p_cb_param)

  Callback function used to provide special measurement results to the application.

Enumerations

- enum as7058_reg_group_ids {
  AS7058_REG_GROUP_ID_PWR = 0,
  AS7058_REG_GROUP_ID_CTRL = 1,
  AS7058_REG_GROUP_ID_LED = 2,
  AS7058_REG_GROUP_ID_PD = 3,
  AS7058_REG_GROUP_ID_IOS = 4,
  AS7058_REG_GROUP_ID_PPG = 5,
  AS7058_REG_GROUP_ID_ECG = 6,
  AS7058_REG_GROUP_ID_SINC = 7,
  AS7058_REG_GROUP_ID_IIR = 8,
  AS7058_REG_GROUP_ID_SEQ = 9,
  AS7058_REG_GROUP_ID_PP = 10,
  AS7058_REG_GROUP_ID_FIFO = 11,
  AS7058_REG_GROUP_ID_NUM = 12 }
- enum as7058_sub_sample_flags {
  AS7058_SUB_SAMPLE_FLAG_NONE = 0x00000000,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 = 0x00000001,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB2 = 0x00000002,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB3 = 0x00000004,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB4 = 0x00000008,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB5 = 0x00000010,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB6 = 0x00000020,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB7 = 0x00000040,
  AS7058_SUB_SAMPLE_FLAG_PPG1_SUB8 = 0x00000080,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 = 0x00000100,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB2 = 0x00000200,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB3 = 0x00000400,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB4 = 0x00000800,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB5 = 0x00001000,
  AS7058_SUB_SAMPLE_FLAG_PPG2_SUB6 = 0x00002000,

AS7058_SUB_SAMPLE_FLAG_PPG2_SUB7 = 0x00004000,
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB8 = 0x00008000,
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ1_SUB1 = 0x00010000,
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ1_SUB2 = 0x00020000,
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ2_SUB1 = 0x00040000 }

- enum as7058_sub_sample_ids {
  AS7058_SUB_SAMPLE_ID_DISABLED = 0,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB1 = 1,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB2 = 2,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB3 = 3,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB4 = 4,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB5 = 5,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB6 = 6,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB7 = 7,
  AS7058_SUB_SAMPLE_ID_PPG1_SUB8 = 8,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB1 = 9,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB2 = 10,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB3 = 11,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB4 = 12,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB5 = 13,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB6 = 14,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB7 = 15,
  AS7058_SUB_SAMPLE_ID_PPG2_SUB8 = 16,
  AS7058_SUB_SAMPLE_ID_ECG_SEQ1_SUB1 = 17,
  AS7058_SUB_SAMPLE_ID_ECG_SEQ1_SUB2 = 18,
  AS7058_SUB_SAMPLE_ID_ECG_SEQ2_SUB1 = 19,
  AS7058_SUB_SAMPLE_ID_NUM }
- enum as7058_modulator {
  AS7058_MODULATOR_PPG1 = 0,
  AS7058_MODULATOR_PPG2 = 1,
  AS7058_MODULATOR_ECG_SEQ1 = 2,
  AS7058_MODULATOR_ECG_SEQ2 = 3,
  AS7058_MODULATOR_NUM_PPG = 2,
  AS7058_MODULATOR_NUM = 4 }
- enum as7058_sub_sample {
  AS7058_MODULATOR_SUB_SAMPLE_1 = 0,
  AS7058_MODULATOR_SUB_SAMPLE_2 = 1,
  AS7058_MODULATOR_SUB_SAMPLE_3 = 2,
  AS7058_MODULATOR_SUB_SAMPLE_4 = 3,
  AS7058_MODULATOR_SUB_SAMPLE_5 = 4,
  AS7058_MODULATOR_SUB_SAMPLE_6 = 5,
  AS7058_MODULATOR_SUB_SAMPLE_7 = 6,
  AS7058_MODULATOR_SUB_SAMPLE_8 = 7,
  AS7058_MODULATOR_SUB_SAMPLE_NUM_PPG = 8,
  AS7058_MODULATOR_SUB_SAMPLE_INVALID = 0xFF }
- enum as7058_fifo_data_marker {
  AS7058_FIFO_DATA_MARKER_PPG1_FIRST = 0,
  AS7058_FIFO_DATA_MARKER_PPG1_OTHER = 1,
  AS7058_FIFO_DATA_MARKER_PPG2_FIRST = 2,
  AS7058_FIFO_DATA_MARKER_PPG2_OTHER = 3,
  AS7058_FIFO_DATA_MARKER_ECG_SEQ1_SUB1 = 4,
  AS7058_FIFO_DATA_MARKER_ECG_SEQ1_SUB2 = 5,
  AS7058_FIFO_DATA_MARKER_ECG_SEQ2_SUB1 = 6,
  AS7058_FIFO_DATA_MARKER_STATUS = 7 }

- enum as7058_meas_mode {
  AS7058_MEAS_MODE_NORMAL = 0,
  AS7058_MEAS_MODE_SPECIAL_BIOZ = 1,
  AS7058_MEAS_MODE_SPECIAL_SCALING_EDA = 2,
  AS7058_MEAS_MODE_SPECIAL_PD_OFFSET_CALIBRATION = 3,
  AS7058_MEAS_MODE_NUM = 4 }
- enum as7058_reg_addresses {
  AS7058_REGADDR_OTP_BIOZ_REF_L = 0x0E,
  AS7058_REGADDR_OTP_BIOZ_REF_H = 0x0F,
  AS7058_REGADDR_OTP_GSR_REF_L = 0x13,
  AS7058_REGADDR_OTP_GSR_REF_H = 0x14,
  AS7058_REGADDR_OTP_TEMP_REF_L = 0x15,
  AS7058_REGADDR_OTP_TEMP_REF_H = 0x16,
  AS7058_REGADDR_CLK_CFG = 0x18,
  AS7058_REGADDR_REF_CFG1 = 0x19,
  AS7058_REGADDR_REF_CFG2 = 0x1A,
  AS7058_REGADDR_REF_CFG3 = 0x1B,
  AS7058_REGADDR_STANDBY_ON1 = 0x1C,
  AS7058_REGADDR_STANDBY_ON2 = 0x1D,
  AS7058_REGADDR_STANDBY_EN1 = 0x1E,
  AS7058_REGADDR_STANDBY_EN2 = 0x1F,
  AS7058_REGADDR_STANDBY_EN3 = 0x20,
  AS7058_REGADDR_STANDBY_EN4 = 0x21,
  AS7058_REGADDR_STANDBY_EN5 = 0x22,
  AS7058_REGADDR_STANDBY_EN6 = 0x23,
  AS7058_REGADDR_STANDBY_EN7 = 0x24,
  AS7058_REGADDR_STANDBY_EN8 = 0x25,
  AS7058_REGADDR_STANDBY_EN9 = 0x26,
  AS7058_REGADDR_STANDBY_EN10 = 0x27,
  AS7058_REGADDR_STANDBY_EN11 = 0x28,
  AS7058_REGADDR_STANDBY_EN12 = 0x29,
  AS7058_REGADDR_STANDBY_EN13 = 0x2A,
  AS7058_REGADDR_STANDBY_EN14 = 0x2B,
  AS7058_REGADDR_PWR_ON = 0x2D,
  AS7058_REGADDR_PWR_ISO = 0x2E,
  AS7058_REGADDR_PWR_STAT = 0x2F,
  AS7058_REGADDR_I2C_MODE = 0x31,
  AS7058_REGADDR_INT_CFG = 0x32,
  AS7058_REGADDR_IF_CFG = 0x33,
  AS7058_REGADDR_GPIO_CFG1 = 0x34,
  AS7058_REGADDR_GPIO_CFG2 = 0x35,
  AS7058_REGADDR_IO_CFG = 0x36,
  AS7058_REGADDR_PPGMOD_CFG1 = 0x37,
  AS7058_REGADDR_PPGMOD_CFG2 = 0x38,
  AS7058_REGADDR_PPGMOD_CFG3 = 0x39,
  AS7058_REGADDR_PPGMOD1_CFG1 = 0x3A,
  AS7058_REGADDR_PPGMOD1_CFG2 = 0x3B,
  AS7058_REGADDR_PPGMOD1_CFG3 = 0x3C,
  AS7058_REGADDR_PPGMOD2_CFG1 = 0x3D,
  AS7058_REGADDR_PPGMOD2_CFG2 = 0x3E,
  AS7058_REGADDR_PPGMOD2_CFG3 = 0x3F,
  AS7058_REGADDR_VCSEL_PASSWORD = 0x40,
  AS7058_REGADDR_VCSEL_CFG = 0x41,
  AS7058_REGADDR_VCSEL_MODE = 0x42,

AS7058_REGADDR_LED_CFG = 0x43,
AS7058_REGADDR_LED_DRV1 = 0x44,
AS7058_REGADDR_LED_DRV2 = 0x45,
AS7058_REGADDR_LED1_ICTRL = 0x46,
AS7058_REGADDR_LED2_ICTRL = 0x47,
AS7058_REGADDR_LED3_ICTRL = 0x48,
AS7058_REGADDR_LED4_ICTRL = 0x49,
AS7058_REGADDR_LED5_ICTRL = 0x4A,
AS7058_REGADDR_LED6_ICTRL = 0x4B,
AS7058_REGADDR_LED7_ICTRL = 0x4C,
AS7058_REGADDR_LED8_ICTRL = 0x4D,
AS7058_REGADDR_LED_IRNG1 = 0x4E,
AS7058_REGADDR_LED_IRNG2 = 0x4F,
AS7058_REGADDR_LED_SUB1 = 0x50,
AS7058_REGADDR_LED_SUB2 = 0x51,
AS7058_REGADDR_LED_SUB3 = 0x52,
AS7058_REGADDR_LED_SUB4 = 0x53,
AS7058_REGADDR_LED_SUB5 = 0x54,
AS7058_REGADDR_LED_SUB6 = 0x55,
AS7058_REGADDR_LED_SUB7 = 0x56,
AS7058_REGADDR_LED_SUB8 = 0x57,
AS7058_REGADDR_LOWVDS_WAIT = 0x58,
AS7058_REGADDR_PDSEL_CFG = 0x59,
AS7058_REGADDR_PPG1_PDSEL1 = 0x5A,
AS7058_REGADDR_PPG1_PDSEL2 = 0x5B,
AS7058_REGADDR_PPG1_PDSEL3 = 0x5C,
AS7058_REGADDR_PPG1_PDSEL4 = 0x5D,
AS7058_REGADDR_PPG1_PDSEL5 = 0x5E,
AS7058_REGADDR_PPG1_PDSEL6 = 0x5F,
AS7058_REGADDR_PPG1_PDSEL7 = 0x60,
AS7058_REGADDR_PPG1_PDSEL8 = 0x61,
AS7058_REGADDR_PPG2_PDSEL1 = 0x62,
AS7058_REGADDR_PPG2_PDSEL2 = 0x63,
AS7058_REGADDR_PPG2_PDSEL3 = 0x64,
AS7058_REGADDR_PPG2_PDSEL4 = 0x65,
AS7058_REGADDR_PPG2_PDSEL5 = 0x66,
AS7058_REGADDR_PPG2_PDSEL6 = 0x67,
AS7058_REGADDR_PPG2_PDSEL7 = 0x68,
AS7058_REGADDR_PPG2_PDSEL8 = 0x69,
AS7058_REGADDR_PPG2_AFESEL1 = 0x6A,
AS7058_REGADDR_PPG2_AFESEL2 = 0x6B,
AS7058_REGADDR_PPG2_AFESEL3 = 0x6C,
AS7058_REGADDR_PPG2_AFESEL4 = 0x6D,
AS7058_REGADDR_PPG2_AFEEN = 0x6E,
AS7058_REGADDR_PPG_SINC_CFGA = 0x6F,
AS7058_REGADDR_PPG_SINC_CFGB = 0x70,
AS7058_REGADDR_PPG_SINC_CFGC = 0x71,
AS7058_REGADDR_PPG_SINC_CFGD = 0x72,
AS7058_REGADDR_ECG1_SINC_CFGA = 0x73,
AS7058_REGADDR_ECG1_SINC_CFGB = 0x74,
AS7058_REGADDR_ECG1_SINC_CFGC = 0x75,
AS7058_REGADDR_ECG2_SINC_CFGA = 0x76,
AS7058_REGADDR_ECG2_SINC_CFGB = 0x77,
AS7058_REGADDR_ECG2_SINC_CFGC = 0x78,

AS7058_REGADDR_ECG_SINC_CFG = 0x79,
AS7058_REGADDR_IOS_PPG1_SUB1 = 0x7A,
AS7058_REGADDR_IOS_PPG1_SUB2 = 0x7B,
AS7058_REGADDR_IOS_PPG1_SUB3 = 0x7C,
AS7058_REGADDR_IOS_PPG1_SUB4 = 0x7D,
AS7058_REGADDR_IOS_PPG1_SUB5 = 0x7E,
AS7058_REGADDR_IOS_PPG1_SUB6 = 0x7F,
AS7058_REGADDR_IOS_PPG1_SUB7 = 0x80,
AS7058_REGADDR_IOS_PPG1_SUB8 = 0x81,
AS7058_REGADDR_IOS_PPG2_SUB1 = 0x82,
AS7058_REGADDR_IOS_PPG2_SUB2 = 0x83,
AS7058_REGADDR_IOS_PPG2_SUB3 = 0x84,
AS7058_REGADDR_IOS_PPG2_SUB4 = 0x85,
AS7058_REGADDR_IOS_PPG2_SUB5 = 0x86,
AS7058_REGADDR_IOS_PPG2_SUB6 = 0x87,
AS7058_REGADDR_IOS_PPG2_SUB7 = 0x88,
AS7058_REGADDR_IOS_PPG2_SUB8 = 0x89,
AS7058_REGADDR_IOS_LEDOFF = 0x8A,
AS7058_REGADDR_IOS_CFG = 0x8B,
AS7058_REGADDR_AOC_SAR_THRES = 0x8C,
AS7058_REGADDR_AOC_SAR_RANGE = 0x8D,
AS7058_REGADDR_AOC_SAR_PPG1 = 0x8E,
AS7058_REGADDR_AOC_SAR_PPG2 = 0x8F,
AS7058_REGADDR_PP_CFG = 0x90,
AS7058_REGADDR_PPG1_PP1 = 0x91,
AS7058_REGADDR_PPG1_PP2 = 0x92,
AS7058_REGADDR_PPG2_PP1 = 0x93,
AS7058_REGADDR_PPG2_PP2 = 0x94,
AS7058_REGADDR_IRQ_ENABLE = 0x95,
AS7058_REGADDR_PPG_SUB_WAIT = 0x96,
AS7058_REGADDR_PPG_SAR_WAIT = 0x97,
AS7058_REGADDR_PPG_LED_INIT = 0x98,
AS7058_REGADDR_PPG_FREQL = 0x99,
AS7058_REGADDR_PPG_FREQH = 0x9A,
AS7058_REGADDR_PPG1_SUB_EN = 0x9B,
AS7058_REGADDR_PPG2_SUB_EN = 0x9C,
AS7058_REGADDR_PPG_MODE1 = 0x9D,
AS7058_REGADDR_PPG_MODE2 = 0x9E,
AS7058_REGADDR_PPG_MODE3 = 0x9F,
AS7058_REGADDR_PPG_MODE4 = 0xA0,
AS7058_REGADDR_PPG_MODE5 = 0xA1,
AS7058_REGADDR_PPG_MODE6 = 0xA2,
AS7058_REGADDR_PPG_MODE7 = 0xA3,
AS7058_REGADDR_PPG_MODE8 = 0xA4,
AS7058_REGADDR_PPG_CFG = 0xA5,
AS7058_REGADDR_ECG_FREQL = 0xA6,
AS7058_REGADDR_ECG_FREQH = 0xA7,
AS7058_REGADDR_ECG1_FREQDIVL = 0xA8,
AS7058_REGADDR_ECG1_FREQDIVH = 0xA9,
AS7058_REGADDR_ECG2_FREQDIVL = 0xAA,
AS7058_REGADDR_ECG2_FREQDIVH = 0xAB,
AS7058_REGADDR_ECG_SUBS = 0xAC,
AS7058_REGADDR_LEADOFF_INITL = 0xAD,
AS7058_REGADDR_LEADOFF_INITH = 0xAE,

AS7058_REGADDR_ECG_INITL = 0xAF,
AS7058_REGADDR_ECG_INITH = 0xB0,
AS7058_REGADDR_SAMPLE_NUM = 0xB1,
AS7058_REGADDR_BIOZ_CFG = 0xB2,
AS7058_REGADDR_BIOZ_EXCIT = 0xB3,
AS7058_REGADDR_BIOZ_MIXER = 0xB4,
AS7058_REGADDR_BIOZ_SELECT = 0xB5,
AS7058_REGADDR_BIOZ_GAIN = 0xB6,
AS7058_REGADDR_ECGMOD_CFG1 = 0xB7,
AS7058_REGADDR_ECGMOD_CFG2 = 0xB8,
AS7058_REGADDR_ECGIMUX_CFG1 = 0xB9,
AS7058_REGADDR_ECGIMUX_CFG2 = 0xBA,
AS7058_REGADDR_ECGIMUX_CFG3 = 0xBB,
AS7058_REGADDR_ECGAMP_CFG1 = 0xBC,
AS7058_REGADDR_ECGAMP_CFG2 = 0xBD,
AS7058_REGADDR_ECGAMP_CFG3 = 0xBE,
AS7058_REGADDR_ECGAMP_CFG4 = 0xBF,
AS7058_REGADDR_ECGAMP_CFG5 = 0xC0,
AS7058_REGADDR_ECGAMP_CFG6 = 0xC1,
AS7058_REGADDR_ECGAMP_CFG7 = 0xC2,
AS7058_REGADDR_ECG_BIOZ = 0xC3,
AS7058_REGADDR_LEADOFF_CFG = 0xC4,
AS7058_REGADDR_LEADOFF_THRESL = 0xC5,
AS7058_REGADDR_LEADOFF_THRESH = 0xC6,
AS7058_REGADDR_IIR_CFG = 0xC7,
AS7058_REGADDR_IIR_COEFF_ADDR = 0xC8,
AS7058_REGADDR_IIR_COEFF_DATA = 0xC9,
AS7058_REGADDR_FIFO_THRESHOLD = 0xCA,
AS7058_REGADDR_FIFO_CTRL = 0xCB,
AS7058_REGADDR_PRODUCT_ID = 0xEB,
AS7058_REGADDR_SILICON_ID = 0xEC,
AS7058_REGADDR_REVISION = 0xED,
AS7058_REGADDR_GPIO_CTRL = 0xEE,
AS7058_REGADDR_CHIP_CTRL = 0xEF,
AS7058_REGADDR_SEQ_START = 0xF0,
AS7058_REGADDR_STATUS_CGB = 0xF1,
AS7058_REGADDR_STATUS_SEQ = 0xF2,
AS7058_REGADDR_STATUS_LED = 0xF3,
AS7058_REGADDR_STATUS_ASATA = 0xF4,
AS7058_REGADDR_STATUS_ASATB = 0xF5,
AS7058_REGADDR_STATUS_VCSEL = 0xF6,
AS7058_REGADDR_STATUS_VCSEL_VSS = 0xF7,
AS7058_REGADDR_STATUS_VCSEL_VDD = 0xF8,
AS7058_REGADDR_STATUS_LEADOFF = 0xF9,
AS7058_REGADDR_STATUS = 0xFA,
AS7058_REGADDR_FIFO_LEVEL0 = 0xFB,
AS7058_REGADDR_FIFO_LEVEL1 = 0xFC,
AS7058_REGADDR_FIFOL = 0xFD,
AS7058_REGADDR_FIFOM = 0xFE,
AS7058_REGADDR_FIFOH = 0xFF }

- enum as7058_bioz_measurement_types {
  AS7058_BIOZ_MEASUREMENT_TYPE_SHORT = 0,
  AS7058_BIOZ_MEASUREMENT_TYPE_RESISTOR = 1,
  AS7058_BIOZ_MEASUREMENT_TYPE_BODY = 2,

AS7058_BIOZ_MEASUREMENT_TYPE_WRIST = 3,
AS7058_BIOZ_MEASUREMENT_TYPE_FINGER = 4,
AS7058_BIOZ_MEASUREMENT_TYPE_TOTAL = 5,
AS7058_BIOZ_MEASUREMENT_TYPE_NUM = 6 }

- enum as7058_sar_transfer_mode {
AS7058_SAR_TRANSFER_MODE_SEPARATE = 0,
AS7058_SAR_TRANSFER_MODE_INLINE }

4.4.1 Detailed Description

Description of the used data types.

These are the type definitions used by AS7058 chip library.

4.4.2 Macro Definition Documentation

4.4.2.1 AS7058_SILICON_ID `#define AS7058_SILICON_ID 0x92`

Silicon ID of AS7058.

4.4.2.2 AS7058_MAX_GROUP_SIZE `#define AS7058_MAX_GROUP_SIZE sizeof(as7058_reg_group_iir_t)`

Maximum combined size of the register values of a single register group.

4.4.2.3 AS7058_FIFO_SAMPLE_SIZE `#define AS7058_FIFO_SAMPLE_SIZE 3`

Size of one sample inside the FIFO.

4.4.2.4 AS7058_FIFO_DATA_BUFFER_SIZE `#define AS7058_FIFO_DATA_BUFFER_SIZE 1536`

Size of the buffer which holds the FIFO data.

4.4.2.5 AS7058_SAMPLE_MAX_VALUE `#define AS7058_SAMPLE_MAX_VALUE 0xFFFFF`

Maximum value of a sample acquired by AS7058.

4.4.2.6 AS7058_PD_OFFSET_CALIBRATION_COMPENSATION_TABLE_ELEMENTS_NUM `#define A↩ S7058_PD_OFFSET_CALIBRATION_COMPENSATION_TABLE_ELEMENTS_NUM 15`

Number of elements in a PD offset calibration compensation table. During PD offset calibration, the SAR-controlled photodiode offset current bits (upper four bits) are manipulated. The remaining bits are left unmodified. A table element is generated for each value of the SAR-controlled bits, except for when the SAR-controlled bits are zero.

4.4.2.7 M_AS7058_SUB_SAMPLE_ID_TO_FLAG `#define M_AS7058_SUB_SAMPLE_ID_TO_FLAG( sub_sample_id ) (as7058_sub_sample_flags_t)(1 << ((sub_sample_id)-1))`

Calculate the sub-sample flag from the ID.

4.4.3 Typedef Documentation

4.4.3.1 as7058_reg_group_ids_t `typedef uint8_t as7058_reg_group_ids_t`

Type of as7058_reg_group_ids.

4.4.3.2 as7058_sub_sample_flags_t `typedef uint32_t as7058_sub_sample_flags_t`

Type of as7058_sub_sample_flags.

4.4.3.3 as7058_sub_sample_ids_t `typedef uint8_t as7058_sub_sample_ids_t`

Type of as7058_sub_sample_ids.

4.4.3.4 as7058_modulator_t `typedef uint8_t as7058_modulator_t`

Type for as7058_modulator.

4.4.3.5 as7058_modulator_sub_sample_t `typedef uint8_t as7058_modulator_sub_sample_t`

Type for as7058_sub_sample.

4.4.3.6 as7058_fifo_data_marker_t `typedef uint8_t as7058_fifo_data_marker_t`

Type of as7058_fifo_data_marker.

4.4.3.7 as7058_meas_mode_t `typedef uint8_t as7058_meas_mode_t`

Type of as7058_meas_mode.

4.4.3.8 as7058_reg_addresses_t `typedef uint8_t as7058_reg_addresses_t`

Type of as7058_reg_addresses.

4.4.3.9 as7058_status_events_t `typedef struct status_regs as7058_status_events_t`

Register group for sensor status events.

4.4.3.10 as7058_sar_transfer_mode_t `typedef uint8_t as7058_sar_transfer_mode_t`

Type for as7058_sar_transfer_mode.

4.4.3.11 as7058_callback_t `typedef void(* as7058_callback_t) (err_code_t error, const uint8_t *p_fifo_data, uint16_t fifo_data_size, const agc_status_t *p_agc_statuses, uint8_t agc_statuses_num, as7058_status_events_t sensor_events, const void *p_cb_param)`

Callback function used to provide regular measurement results to the application.

A callback of this type is registered via the function as7058_initialize. During measurements of mode AS7058_MEAS_MODE_NORMAL, this function is called whenever data is received from AS7058.

Parameters

in	*error*	Default ERR_SUCCESS, otherwise an error occurred during measurement and the measurement is stopped. See error_codes.
in	*p_fifo_data*	Pointer to the measurement data, the content depends on configuration.
in	*fifo_data_size*	Size of the FIFO data buffer in bytes.
in	*p_agc_statuses*	Pointer to AGC status structure.
in	*agc_statuses_num*	Number of AGC status elements.
in	*sensor_events*	Combination of all possible sensor status events but FIFO status. See as7058_status_events_t. When an interrupt is disabled, the corresponding status field is always set to zero.
in	*p_cb_param*	Application parameter which was set when as7058_initialize was called.

4.4.3.12 as7058_callback_special_measurement_t `typedef void(* as7058_callback_special_measurement_t) (as7058_meas_mode_t mode, const as7058_special_measurement_result_t *p_result, uint16_t result_size, const void *p_cb_param)`

Callback function used to provide special measurement results to the application.

A callback of this type is registered via the function as7058_initialize. During measurements of modes AS7058_MEAS_MODE_SPECIAL_BIOZ and AS7058_MEAS_MODE_SPECIAL_SCALING_EDA, this function is called whenever measurement results are available.

Parameters

in	*mode*	Current measurement mode.
in	*p_result*	Result of the special measurement. See as7058_special_measurement_result_t.
in	*result_size*	Size of the special measurement result.
in	*p_cb_param*	Application parameter which was set when as7058_initialize was called.

4.4.4 Enumeration Type Documentation

4.4.4.1 as7058_reg_group_ids `enum as7058_reg_group_ids`

Definition of the register group IDs.

Enumerator

AS7058_REG_GROUP_ID_PWR	ID of the `power` register group. See as7058_reg_group_power_t for the contents of the group.
AS7058_REG_GROUP_ID_CTRL	ID of the `control` register group. See as7058_reg_group_control_t for the contents of the group.
AS7058_REG_GROUP_ID_LED	ID of the `led` register group. See as7058_reg_group_led_t for the contents of the group.
AS7058_REG_GROUP_ID_PD	ID of the `pd` register group. See as7058_reg_group_pd_t for the contents of the group.
AS7058_REG_GROUP_ID_IOS	ID of the `ios` register group. See as7058_reg_group_ios_t for the contents of the group.
AS7058_REG_GROUP_ID_PPG	ID of the `ppg` register group. See as7058_reg_group_ppg_t for the contents of the group.
AS7058_REG_GROUP_ID_ECG	ID of the `ecg` register group. See as7058_reg_group_ecg_t for the contents of the group.
AS7058_REG_GROUP_ID_SINC	ID of the `sinc` register group. See as7058_reg_group_sinc_t for the contents of the group.
AS7058_REG_GROUP_ID_IIR	ID of the `iir` register group. See as7058_reg_group_iir_t for the contents of the group.
AS7058_REG_GROUP_ID_SEQ	ID of the `seq` register group. See as7058_reg_group_seq_t for the contents of the group.
AS7058_REG_GROUP_ID_PP	ID of the `post` register group. See as7058_reg_group_pp_t for the contents of the group.
AS7058_REG_GROUP_ID_FIFO	ID of the `fifo` register group. See as7058_reg_group_fifo_t for the contents of the group.
AS7058_REG_GROUP_ID_NUM	Number of supported register groups.

4.4.4.2 as7058_sub_sample_flags `enum as7058_sub_sample_flags`

Sub-sample flags.

Enumerator

AS7058_SUB_SAMPLE_FLAG_NONE	No sub-sample is selected.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1	Flag for sub-sample 1 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB2	Flag for sub-sample 2 of PPG modulator 1.

Enumerator

AS7058_SUB_SAMPLE_FLAG_PPG1_SUB3	Flag for sub-sample 3 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB4	Flag for sub-sample 4 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB5	Flag for sub-sample 5 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB6	Flag for sub-sample 6 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB7	Flag for sub-sample 7 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG1_SUB8	Flag for sub-sample 8 of PPG modulator 1.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1	Flag for sub-sample 1 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB2	Flag for sub-sample 2 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB3	Flag for sub-sample 3 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB4	Flag for sub-sample 4 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB5	Flag for sub-sample 5 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB6	Flag for sub-sample 6 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB7	Flag for sub-sample 7 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_PPG2_SUB8	Flag for sub-sample 8 of PPG modulator 2.
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ1_SUB1	Flag for sub-sample 1 of sequence 1 of the ECG modulator.
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ1_SUB2	Flag for sub-sample 2 of sequence 1 of the ECG modulator.
AS7058_SUB_SAMPLE_FLAG_ECG_SEQ2_SUB1	Flag for sub-sample 1 of sequence 2 of the ECG modulator.

4.4.4.3 as7058_sub_sample_ids `enum as7058_sub_sample_ids`

Sub-sample ID.

Enumerator

AS7058_SUB_SAMPLE_ID_DISABLED	Sub-sample is disabled.
AS7058_SUB_SAMPLE_ID_PPG1_SUB1	Sub-sample ID of sub-sample 1 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB2	Sub-sample ID of sub-sample 2 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB3	Sub-sample ID of sub-sample 3 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB4	Sub-sample ID of sub-sample 4 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB5	Sub-sample ID of sub-sample 5 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB6	Sub-sample ID of sub-sample 6 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB7	Sub-sample ID of sub-sample 7 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG1_SUB8	Sub-sample ID of sub-sample 8 of PPG modulator 1.
AS7058_SUB_SAMPLE_ID_PPG2_SUB1	Sub-sample ID of sub-sample 1 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB2	Sub-sample ID of sub-sample 2 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB3	Sub-sample ID of sub-sample 3 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB4	Sub-sample ID of sub-sample 4 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB5	Sub-sample ID of sub-sample 5 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB6	Sub-sample ID of sub-sample 6 of PPG modulator 2.

Enumerator

AS7058_SUB_SAMPLE_ID_PPG2_SUB7	Sub-sample ID of sub-sample 7 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_PPG2_SUB8	Sub-sample ID of sub-sample 8 of PPG modulator 2.
AS7058_SUB_SAMPLE_ID_ECG_SEQ1_SUB1	Sub-sample ID of sub-sample 1 of sequence 1 of the ECG modulator.
AS7058_SUB_SAMPLE_ID_ECG_SEQ1_SUB2	Sub-sample ID of sub-sample 2 of sequence 1 of the ECG modulator.
AS7058_SUB_SAMPLE_ID_ECG_SEQ2_SUB1	Sub-sample ID of sub-sample 1 of sequence 2 of the ECG modulator.
AS7058_SUB_SAMPLE_ID_NUM	Number of supported sub-sample IDs. The last valid ID is AS7058_SUB_SAMPLE_ID_ECG_SEQ2_SUB1.

4.4.4.4 as7058_modulator `enum as7058_modulator`

Identifies the modulators of the AS7058 AFE. Note that if a modulator supports multiple sequences, this enumeration contains separate enumeration constants for each sequence.

Enumerator

AS7058_MODULATOR_PPG1	PPG modulator 1.
AS7058_MODULATOR_PPG2	PPG modulator 2.
AS7058_MODULATOR_ECG_SEQ1	ECG modulator, sequence 1.
AS7058_MODULATOR_ECG_SEQ2	ECG modulator, sequence 2.
AS7058_MODULATOR_NUM_PPG	Number of PPG modulators.
AS7058_MODULATOR_NUM	Total number of modulators.

4.4.4.5 as7058_sub_sample `enum as7058_sub_sample`

Identifies the sub-samples of an as7058_modulator. Note that not all modulators support the same number of sub-samples.

Enumerator

AS7058_MODULATOR_SUB_SAMPLE_1	Sub-sample 1.
AS7058_MODULATOR_SUB_SAMPLE_2	Sub-sample 2.
AS7058_MODULATOR_SUB_SAMPLE_3	Sub-sample 3.
AS7058_MODULATOR_SUB_SAMPLE_4	Sub-sample 4.
AS7058_MODULATOR_SUB_SAMPLE_5	Sub-sample 5.
AS7058_MODULATOR_SUB_SAMPLE_6	Sub-sample 6.
AS7058_MODULATOR_SUB_SAMPLE_7	Sub-sample 7.
AS7058_MODULATOR_SUB_SAMPLE_8	Sub-sample 8.

Enumerator

AS7058_MODULATOR_SUB_SAMPLE_NUM_PPG	Number of sub-samples supported by PPG modulators (AS7058_MODULATOR_PPG1 and AS7058_MODULATOR_PPG2).
AS7058_MODULATOR_SUB_SAMPLE_INVALID	Represents an invalid value.

4.4.4.6 as7058_fifo_data_marker `enum as7058_fifo_data_marker`

First three bits of the FIFO data, which describes the type of the sample.

Enumerator

AS7058_FIFO_DATA_MARKER_PPG1_FIRST	Defines first sub-sample of modulator 1.
AS7058_FIFO_DATA_MARKER_PPG1_OTHER	Other sub-samples of modulator 1.
AS7058_FIFO_DATA_MARKER_PPG2_FIRST	Defines first sub-sample of modulator 2.
AS7058_FIFO_DATA_MARKER_PPG2_OTHER	Other sub-samples of modulator 2.
AS7058_FIFO_DATA_MARKER_ECG_SEQ1_SUB1	Sub-sample 1 of modulator 3 and sequence 1.
AS7058_FIFO_DATA_MARKER_ECG_SEQ1_SUB2	Sub-sample 2 of modulator 3 and sequence 1.
AS7058_FIFO_DATA_MARKER_ECG_SEQ2_SUB1	Sub-sample 1 of modulator 3 and sequence 2.
AS7058_FIFO_DATA_MARKER_STATUS	Status marker of SAR information.

4.4.4.7 as7058_meas_mode `enum as7058_meas_mode`

Measurement modes.

Enumerator

AS7058_MEAS_MODE_NORMAL	Default measurement mode for regular FIFO data acquisition.
AS7058_MEAS_MODE_SPECIAL_BIOZ	Special measurement mode for BioZ.
AS7058_MEAS_MODE_SPECIAL_SCALING_EDA	Special measurement mode to obtain impedance reference values for EDA.
AS7058_MEAS_MODE_SPECIAL_PD_OFFSET_↩ CALIBRATION	Special measurement mode to perform PD offset calibration.
AS7058_MEAS_MODE_NUM	Number of supported measurement modes.

4.4.4.8 as7058_reg_addresses `enum as7058_reg_addresses`

Register definition of AS7058.

Enumerator

AS7058_REGADDR_OTP_BIOZ_REF_L	Address of register OTP14.
AS7058_REGADDR_OTP_BIOZ_REF_H	Address of register OTP15.
AS7058_REGADDR_OTP_GSR_REF_L	Address of register OTP19.
AS7058_REGADDR_OTP_GSR_REF_H	Address of register OTP20.
AS7058_REGADDR_OTP_TEMP_REF_L	Address of register OTP21.
AS7058_REGADDR_OTP_TEMP_REF_H	Address of register OTP22.
AS7058_REGADDR_CLK_CFG	Address of register CLK_CFG.
AS7058_REGADDR_REF_CFG1	Address of register REF_CFG1.
AS7058_REGADDR_REF_CFG2	Address of register REF_CFG2.
AS7058_REGADDR_REF_CFG3	Address of register REF_CFG3.
AS7058_REGADDR_STANDBY_ON1	Address of register STANDBY_ON1.
AS7058_REGADDR_STANDBY_ON2	Address of register STANDBY_ON2.
AS7058_REGADDR_STANDBY_EN1	Address of register STANDBY_EN1.
AS7058_REGADDR_STANDBY_EN2	Address of register STANDBY_EN2.
AS7058_REGADDR_STANDBY_EN3	Address of register STANDBY_EN3.
AS7058_REGADDR_STANDBY_EN4	Address of register STANDBY_EN4.
AS7058_REGADDR_STANDBY_EN5	Address of register STANDBY_EN5.
AS7058_REGADDR_STANDBY_EN6	Address of register STANDBY_EN6.
AS7058_REGADDR_STANDBY_EN7	Address of register STANDBY_EN7.
AS7058_REGADDR_STANDBY_EN8	Address of register STANDBY_EN8.
AS7058_REGADDR_STANDBY_EN9	Address of register STANDBY_EN9.
AS7058_REGADDR_STANDBY_EN10	Address of register STANDBY_EN10.
AS7058_REGADDR_STANDBY_EN11	Address of register STANDBY_EN11.
AS7058_REGADDR_STANDBY_EN12	Address of register STANDBY_EN12.
AS7058_REGADDR_STANDBY_EN13	Address of register STANDBY_EN13.
AS7058_REGADDR_STANDBY_EN14	Address of register STANDBY_EN14.
AS7058_REGADDR_PWR_ON	Address of register PWR_ON.
AS7058_REGADDR_PWR_ISO	Address of register PWR_ISO.
AS7058_REGADDR_PWR_STAT	Address of register PWR_STAT.
AS7058_REGADDR_I2C_MODE	Address of register I2C_MODE.
AS7058_REGADDR_INT_CFG	Address of register INT_CFG.
AS7058_REGADDR_IF_CFG	Address of register IF_CFG.
AS7058_REGADDR_GPIO_CFG1	Address of register GPIO_CFG1.
AS7058_REGADDR_GPIO_CFG2	Address of register GPIO_CFG2.
AS7058_REGADDR_IO_CFG	Address of register IO_CFG.
AS7058_REGADDR_PPGMOD_CFG1	Address of register PPGMOD_CFG1.
AS7058_REGADDR_PPGMOD_CFG2	Address of register PPGMOD_CFG2.
AS7058_REGADDR_PPGMOD_CFG3	Address of register PPGMOD_CFG3.
AS7058_REGADDR_PPGMOD1_CFG1	Address of register PPGMOD1_CFG1.
AS7058_REGADDR_PPGMOD1_CFG2	Address of register PPGMOD1_CFG2.
AS7058_REGADDR_PPGMOD1_CFG3	Address of register PPGMOD1_CFG3.
AS7058_REGADDR_PPGMOD2_CFG1	Address of register PPGMOD2_CFG1.
AS7058_REGADDR_PPGMOD2_CFG2	Address of register PPGMOD2_CFG2.

Enumerator

AS7058_REGADDR_PPGMOD2_CFG3	Address of register PPGMOD2_CFG3.
AS7058_REGADDR_VCSEL_PASSWORD	Address of register VCSEL_PASSWORD.
AS7058_REGADDR_VCSEL_CFG	Address of register VCSEL_CFG.
AS7058_REGADDR_VCSEL_MODE	Address of register VCSEL_MODE.
AS7058_REGADDR_LED_CFG	Address of register LED_CFG.
AS7058_REGADDR_LED_DRV1	Address of register LED_DRV1.
AS7058_REGADDR_LED_DRV2	Address of register LED_DRV2.
AS7058_REGADDR_LED1_ICTRL	Address of register LED1_ICTRL.
AS7058_REGADDR_LED2_ICTRL	Address of register LED2_ICTRL.
AS7058_REGADDR_LED3_ICTRL	Address of register LED3_ICTRL.
AS7058_REGADDR_LED4_ICTRL	Address of register LED4_ICTRL.
AS7058_REGADDR_LED5_ICTRL	Address of register LED5_ICTRL.
AS7058_REGADDR_LED6_ICTRL	Address of register LED6_ICTRL.
AS7058_REGADDR_LED7_ICTRL	Address of register LED7_ICTRL.
AS7058_REGADDR_LED8_ICTRL	Address of register LED8_ICTRL.
AS7058_REGADDR_LED_IRNG1	Address of register LED_IRNG1.
AS7058_REGADDR_LED_IRNG2	Address of register LED_IRNG2.
AS7058_REGADDR_LED_SUB1	Address of register LED_SUB1.
AS7058_REGADDR_LED_SUB2	Address of register LED_SUB2.
AS7058_REGADDR_LED_SUB3	Address of register LED_SUB3.
AS7058_REGADDR_LED_SUB4	Address of register LED_SUB4.
AS7058_REGADDR_LED_SUB5	Address of register LED_SUB5.
AS7058_REGADDR_LED_SUB6	Address of register LED_SUB6.
AS7058_REGADDR_LED_SUB7	Address of register LED_SUB7.
AS7058_REGADDR_LED_SUB8	Address of register LED_SUB8.
AS7058_REGADDR_LOWVDS_WAIT	Address of register LOWVDS_WAIT.
AS7058_REGADDR_PDSEL_CFG	Address of register PDSEL_CFG.
AS7058_REGADDR_PPG1_PDSEL1	Address of register PPG1_PDSEL1.
AS7058_REGADDR_PPG1_PDSEL2	Address of register PPG1_PDSEL2.
AS7058_REGADDR_PPG1_PDSEL3	Address of register PPG1_PDSEL3.
AS7058_REGADDR_PPG1_PDSEL4	Address of register PPG1_PDSEL4.
AS7058_REGADDR_PPG1_PDSEL5	Address of register PPG1_PDSEL5.
AS7058_REGADDR_PPG1_PDSEL6	Address of register PPG1_PDSEL6.
AS7058_REGADDR_PPG1_PDSEL7	Address of register PPG1_PDSEL7.
AS7058_REGADDR_PPG1_PDSEL8	Address of register PPG1_PDSEL8.
AS7058_REGADDR_PPG2_PDSEL1	Address of register PPG2_PDSEL1.
AS7058_REGADDR_PPG2_PDSEL2	Address of register PPG2_PDSEL2.
AS7058_REGADDR_PPG2_PDSEL3	Address of register PPG2_PDSEL3.
AS7058_REGADDR_PPG2_PDSEL4	Address of register PPG2_PDSEL4.
AS7058_REGADDR_PPG2_PDSEL5	Address of register PPG2_PDSEL2.
AS7058_REGADDR_PPG2_PDSEL6	Address of register PPG2_PDSEL5.
AS7058_REGADDR_PPG2_PDSEL7	Address of register PPG2_PDSEL6.
AS7058_REGADDR_PPG2_PDSEL8	Address of register PPG2_PDSEL7.

Enumerator

AS7058_REGADDR_PPG2_AFESEL1	Address of register PPG2_AFESEL1.
AS7058_REGADDR_PPG2_AFESEL2	Address of register PPG2_AFESEL2.
AS7058_REGADDR_PPG2_AFESEL3	Address of register PPG2_AFESEL3.
AS7058_REGADDR_PPG2_AFESEL4	Address of register PPG2_AFESEL4.
AS7058_REGADDR_PPG2_AFEEN	Address of register PPG2_AFEEN.
AS7058_REGADDR_PPG_SINC_CFGA	Address of register PPG_SINC_CFGA.
AS7058_REGADDR_PPG_SINC_CFGB	Address of register PPG_SINC_CFGB.
AS7058_REGADDR_PPG_SINC_CFGC	Address of register PPG_SINC_CFGC.
AS7058_REGADDR_PPG_SINC_CFGD	Address of register PPG_SINC_CFGD.
AS7058_REGADDR_ECG1_SINC_CFGA	Address of register ECG1_SINC_CFGA.
AS7058_REGADDR_ECG1_SINC_CFGB	Address of register ECG1_SINC_CFGB.
AS7058_REGADDR_ECG1_SINC_CFGC	Address of register ECG1_SINC_CFGC.
AS7058_REGADDR_ECG2_SINC_CFGA	Address of register ECG2_SINC_CFGA.
AS7058_REGADDR_ECG2_SINC_CFGB	Address of register ECG2_SINC_CFGB.
AS7058_REGADDR_ECG2_SINC_CFGC	Address of register ECG2_SINC_CFGC.
AS7058_REGADDR_ECG_SINC_CFG	Address of register ECG_SINC_CFG.
AS7058_REGADDR_IOS_PPG1_SUB1	Address of register IOS_PPG1_SUB1.
AS7058_REGADDR_IOS_PPG1_SUB2	Address of register IOS_PPG1_SUB2.
AS7058_REGADDR_IOS_PPG1_SUB3	Address of register IOS_PPG1_SUB3.
AS7058_REGADDR_IOS_PPG1_SUB4	Address of register IOS_PPG1_SUB4.
AS7058_REGADDR_IOS_PPG1_SUB5	Address of register IOS_PPG1_SUB5.
AS7058_REGADDR_IOS_PPG1_SUB6	Address of register IOS_PPG1_SUB6.
AS7058_REGADDR_IOS_PPG1_SUB7	Address of register IOS_PPG1_SUB7.
AS7058_REGADDR_IOS_PPG1_SUB8	Address of register IOS_PPG1_SUB8.
AS7058_REGADDR_IOS_PPG2_SUB1	Address of register IOS_PPG2_SUB1.
AS7058_REGADDR_IOS_PPG2_SUB2	Address of register IOS_PPG2_SUB2.
AS7058_REGADDR_IOS_PPG2_SUB3	Address of register IOS_PPG2_SUB3.
AS7058_REGADDR_IOS_PPG2_SUB4	Address of register IOS_PPG2_SUB4.
AS7058_REGADDR_IOS_PPG2_SUB5	Address of register IOS_PPG2_SUB5.
AS7058_REGADDR_IOS_PPG2_SUB6	Address of register IOS_PPG2_SUB6.
AS7058_REGADDR_IOS_PPG2_SUB7	Address of register IOS_PPG2_SUB7.
AS7058_REGADDR_IOS_PPG2_SUB8	Address of register IOS_PPG2_SUB8.
AS7058_REGADDR_IOS_LEDOFF	Address of register IOS_LEDOFF.
AS7058_REGADDR_IOS_CFG	Address of register IOS_CFG.
AS7058_REGADDR_AOC_SAR_THRES	Address of register AOC_SAR_THRES.
AS7058_REGADDR_AOC_SAR_RANGE	Address of register AOC_SAR_RANGE.
AS7058_REGADDR_AOC_SAR_PPG1	Address of register AOC_SAR_PPG1.
AS7058_REGADDR_AOC_SAR_PPG2	Address of register AOC_SAR_PPG2.
AS7058_REGADDR_PP_CFG	Address of register PP_CFG.
AS7058_REGADDR_PPG1_PP1	Address of register PPG1_PP1.
AS7058_REGADDR_PPG1_PP2	Address of register PPG1_PP2.
AS7058_REGADDR_PPG2_PP1	Address of register PPG2_PP1.
AS7058_REGADDR_PPG2_PP2	Address of register PPG2_PP2.

Enumerator

AS7058_REGADDR_IRQ_ENABLE	Address of register IRQ_ENABLE.
AS7058_REGADDR_PPG_SUB_WAIT	Address of register PPG_SUB_WAIT.
AS7058_REGADDR_PPG_SAR_WAIT	Address of register AOC_SAR_PPG2.
AS7058_REGADDR_PPG_LED_INIT	Address of register PPG_LED_INIT.
AS7058_REGADDR_PPG_FREQL	Address of register PPG_FREQL.
AS7058_REGADDR_PPG_FREQH	Address of register PPG_FREQH.
AS7058_REGADDR_PPG1_SUB_EN	Address of register PPG1_SUB_EN.
AS7058_REGADDR_PPG2_SUB_EN	Address of register PPG2_SUB_EN.
AS7058_REGADDR_PPG_MODE1	Address of register PPG_MODE1.
AS7058_REGADDR_PPG_MODE2	Address of register PPG_MODE2.
AS7058_REGADDR_PPG_MODE3	Address of register PPG_MODE3.
AS7058_REGADDR_PPG_MODE4	Address of register PPG_MODE4.
AS7058_REGADDR_PPG_MODE5	Address of register PPG_MODE5.
AS7058_REGADDR_PPG_MODE6	Address of register PPG_MODE6.
AS7058_REGADDR_PPG_MODE7	Address of register PPG_MODE7.
AS7058_REGADDR_PPG_MODE8	Address of register PPG_MODE8.
AS7058_REGADDR_PPG_CFG	Address of register PPG_CFG.
AS7058_REGADDR_ECG_FREQL	Address of register ECG_FREQL.
AS7058_REGADDR_ECG_FREQH	Address of register ECG_FREQH.
AS7058_REGADDR_ECG1_FREQDIVL	Address of register ECG1_FREQDIVL.
AS7058_REGADDR_ECG1_FREQDIVH	Address of register ECG1_FREQDIVH.
AS7058_REGADDR_ECG2_FREQDIVL	Address of register ECG2_FREQDIVL.
AS7058_REGADDR_ECG2_FREQDIVH	Address of register ECG2_FREQDIVH.
AS7058_REGADDR_ECG_SUBS	Address of register ECG_SUBS.
AS7058_REGADDR_LEADOFF_INITL	Address of register LEADOFF_INITL.
AS7058_REGADDR_LEADOFF_INITH	Address of register LEADOFF_INITH.
AS7058_REGADDR_ECG_INITL	Address of register ECG_INITL.
AS7058_REGADDR_ECG_INITH	Address of register ECG_INITH.
AS7058_REGADDR_SAMPLE_NUM	Address of register SAMPLE_NUM.
AS7058_REGADDR_BIOZ_CFG	Address of register BIOZ_CFG.
AS7058_REGADDR_BIOZ_EXCIT	Address of register BIOZ_EXCIT.
AS7058_REGADDR_BIOZ_MIXER	Address of register BIOZ_MIXER.
AS7058_REGADDR_BIOZ_SELECT	Address of register BIOZ_SELECT.
AS7058_REGADDR_BIOZ_GAIN	Address of register BIOZ_GAIN.
AS7058_REGADDR_ECGMOD_CFG1	Address of register ECGMOD_CFG1.
AS7058_REGADDR_ECGMOD_CFG2	Address of register ECGMOD_CFG2.
AS7058_REGADDR_ECGIMUX_CFG1	Address of register ECGIMUX_CFG1.
AS7058_REGADDR_ECGIMUX_CFG2	Address of register ECGIMUX_CFG2.
AS7058_REGADDR_ECGIMUX_CFG3	Address of register ECGIMUX_CFG3.
AS7058_REGADDR_ECGAMP_CFG1	Address of register ECGAMP_CFG1.
AS7058_REGADDR_ECGAMP_CFG2	Address of register ECGAMP_CFG2.
AS7058_REGADDR_ECGAMP_CFG3	Address of register ECGAMP_CFG3.
AS7058_REGADDR_ECGAMP_CFG4	Address of register ECGAMP_CFG4.

Enumerator

AS7058_REGADDR_ECGAMP_CFG5	Address of register ECGAMP_CFG5.
AS7058_REGADDR_ECGAMP_CFG6	Address of register ECGAMP_CFG6.
AS7058_REGADDR_ECGAMP_CFG7	Address of register ECGAMP_CFG7.
AS7058_REGADDR_ECG_BIOZ	Address of register ECG_BIOZ.
AS7058_REGADDR_LEADOFF_CFG	Address of register LEADOFF_CFG.
AS7058_REGADDR_LEADOFF_THRESL	Address of register LEADOFF_THRESL.
AS7058_REGADDR_LEADOFF_THRESH	Address of register LEADOFF_THRESH.
AS7058_REGADDR_IIR_CFG	Address of register IIR_CFG.
AS7058_REGADDR_IIR_COEFF_ADDR	Address of register IIR_COEFF_ADDR.
AS7058_REGADDR_IIR_COEFF_DATA	Address of register IIR_COEFF_DATA.
AS7058_REGADDR_FIFO_THRESHOLD	Address of register FIFO_THRESHOLD.
AS7058_REGADDR_FIFO_CTRL	Address of register FIFO_CTRL.
AS7058_REGADDR_PRODUCT_ID	Address of register PRODUCT_ID.
AS7058_REGADDR_SILICON_ID	Address of register SILICON_ID.
AS7058_REGADDR_REVISION	Address of register REVISION.
AS7058_REGADDR_GPIO_CTRL	Address of register GPIO_CTRL.
AS7058_REGADDR_CHIP_CTRL	Address of register CHIP_CTRL.
AS7058_REGADDR_SEQ_START	Address of register SEQ_START.
AS7058_REGADDR_STATUS_CGB	Address of register STATUS_CGBA.
AS7058_REGADDR_STATUS_SEQ	Address of register STATUS_SEQ.
AS7058_REGADDR_STATUS_LED	Address of register STATUS_LED.
AS7058_REGADDR_STATUS_ASATA	Address of register STATUS_ASATA.
AS7058_REGADDR_STATUS_ASATB	Address of register STATUS_ASATB.
AS7058_REGADDR_STATUS_VCSEL	Address of register STATUS_VCSEL.
AS7058_REGADDR_STATUS_VCSEL_VSS	Address of register STATUS_VCSEL_VSS.
AS7058_REGADDR_STATUS_VCSEL_VDD	Address of register STATUS_VCSEL_VDD.
AS7058_REGADDR_STATUS_LEADOFF	Address of register STATUS_LEADOFF.
AS7058_REGADDR_STATUS	Address of register STATUS.
AS7058_REGADDR_FIFO_LEVEL0	Address of register FIFO_LEVEL0.
AS7058_REGADDR_FIFO_LEVEL1	Address of register FIFO_LEVEL1.
AS7058_REGADDR_FIFOL	Address of register FIFOL.
AS7058_REGADDR_FIFOM	Address of register FIFOM.
AS7058_REGADDR_FIFOH	Address of register FIFOH.

4.4.4.9 as7058_bioz_measurement_types `enum as7058_bioz_measurement_types`

BioZ measurement types

Enumerator

AS7058_BIOZ_MEASUREMENT_TYPE_SHORT	Measurement of a short circuit.

Enumerator

AS7058_BIOZ_MEASUREMENT_TYPE_RESISTOR	Measurement of the internal 2k reference resistor.
AS7058_BIOZ_MEASUREMENT_TYPE_BODY	Measurement of the body impedance.
AS7058_BIOZ_MEASUREMENT_TYPE_WRIST	Measurement of the wrist impedance.
AS7058_BIOZ_MEASUREMENT_TYPE_FINGER	Measurement of the finger impedance.
AS7058_BIOZ_MEASUREMENT_TYPE_TOTAL	Measurement of the total impedance.
AS7058_BIOZ_MEASUREMENT_TYPE_NUM	Number of measurement types.

4.4.4.10 as7058_sar_transfer_mode `enum as7058_sar_transfer_mode`

Represents how the AS7058 AFE provides the SAR status.

Enumerator

AS7058_SAR_TRANSFER_MODE_SEPARATE	PD offset is provided in a separate FIFO item.
AS7058_SAR_TRANSFER_MODE_INLINE	ADC data and upper four bits of the PD offset are contained in a single FIFO item. The AS7058 AFE truncates the ADC data to make room for the upper four bits of the PD offset. The lower four bits of the PD offset (which are not controlled by SAR) must be known from configuration or must be read from the corresponding register.

4.5 Standard Includes

Description of standard includes.

Macros

- #define TRUE 1
- #define FALSE 0

4.5.1 Detailed Description

Description of standard includes.

Here you can find generic standard includes, which are used by ams projects.

4.5.2 Macro Definition Documentation

4.5.2.1 TRUE `#define TRUE 1`

Boolean representation of 'true' to be used with flat uint8_t type

4.5.2.2 FALSE `#define FALSE 0`

Boolean representation of 'false' to be used with flat uint8_t type

4.6 Vital Signs AGC

Data Structures

- struct agc_status_t
- struct agc_configuration_t
- struct agc_version_t

Macros

- #define AGC_MAX_CHANNEL_CNT 4

Typedefs

- typedef uint8_t agc_change_state_t
- typedef uint8_t agc_mode_t
- typedef uint8_t agc_ampl_cntl_mode_t
- typedef uint8_t agc_channel_id_t

Enumerations

- enum agc_change_state {
  AGC_STATE_UNCHANGED = 0,
  AGC_STATE_INCREASED = 1,
  AGC_STATE_DECREASED = 2,
  AGC_STATE_ATMIN = 3,
  AGC_STATE_ATMAX = 4,
  AGC_STATE_NOT_CONTROLLED = 5,
  AGC_STATE_UNDEFINED = 6 }
- enum agc_mode {
  AGC_MODE_DEFAULT = 0,
  AGC_MODE_STATUS = 1 }
- enum agc_ampl_cntl_mode {
  AGC_AMPL_CNTL_MODE_DISABLED = 0,
  AGC_AMPL_CNTL_MODE_AUTO = 1,
  AGC_AMPL_CNTL_MODE_EXTERNAL = 2 }

4.6.1 Detailed Description

4.6.2 Macro Definition Documentation

4.6.2.1 AGC_MAX_CHANNEL_CNT `#define AGC_MAX_CHANNEL_CNT 4`

The maximum number of enabled AGC instances.

4.6.3 Typedef Documentation

4.6.3.1 agc_change_state_t `typedef uint8_t agc_change_state_t`

Type for agc_change_state

4.6.3.2 agc_mode_t `typedef uint8_t agc_mode_t`

Type for agc_mode.

4.6.3.3 agc_ampl_cntl_mode_t `typedef uint8_t agc_ampl_cntl_mode_t`

Type for agc_ampl_cntl_mode.

4.6.3.4 agc_channel_id_t `typedef uint8_t agc_channel_id_t`

Represents a PPG channel. A value of 0 represents a disabled channels, all other values are chip-specific.

4.6.4 Enumeration Type Documentation

4.6.4.1 agc_change_state `enum agc_change_state`

AGC current change information.

Enumerator

AGC_STATE_UNCHANGED	Current not changed.
AGC_STATE_INCREASED	Current increased.
AGC_STATE_DECREASED	Current decreased.
AGC_STATE_ATMIN	Current needs to be decreased, but is at the lower limit.
AGC_STATE_ATMAX	Current needs to be increased, but is at the upper limit.
AGC_STATE_NOT_CONTROLLED	Current is not controlled as amplitude control is disabled.
AGC_STATE_UNDEFINED	Current is undefined. For example the channel has no connected LED.

4.6.4.2 agc_mode `enum agc_mode`

Operating modes of the AGC algorithm.

Enumerator

AGC_MODE_DEFAULT	Default mode.
AGC_MODE_STATUS	AGC algorithm is disabled but returns at least one status package for the selected channel.

4.6.4.3 agc_ampl_cntl_mode `enum agc_ampl_cntl_mode`

Amplitude control modes of the AGC algorithm.

Enumerator

AGC_AMPL_CNTL_MODE_DISABLED	No amplitude control enabled.
AGC_AMPL_CNTL_MODE_AUTO	Internal amplitude control enabled.
AGC_AMPL_CNTL_MODE_EXTERNAL	Amplitude is controlled by an external algorithm.

4.7 Sub-sample extraction

Extract module to get dedicated sub-samples from FIFO data stream.

Data Structures

- struct extract_metadata
- struct as7058_extract_metadata_t

Functions

- err_code_t as7058_extract_samples (as7058_sub_sample_ids_t extract_sub_id, const uint8_t *p_fifo_data, uint16_t fifo_data_size, uint32_t *p_chan_data, uint16_t *p_chan_data_num, as7058_extract_metadata_t *p_metadata)

  Extract the requested sub-samples from FIFO data stream.

4.7.1 Detailed Description

Extract module to get dedicated sub-samples from FIFO data stream.

4.7.2 Function Documentation

4.7.2.1 as7058_extract_samples()

```
err_code_t as7058_extract_samples (
        as7058_sub_sample_ids_t extract_sub_id,
        const uint8_t * p_fifo_data,
        uint16_t fifo_data_size,
        uint32_t * p_chan_data,
        uint16_t * p_chan_data_num,
        as7058_extract_metadata_t * p_metadata )
```

Extract the requested sub-samples from FIFO data stream.

Parameters

in	*extract_sub_id*	Sub-sample ID for which the FIFO data shall extracted.
in	*p_fifo_data*	FIFO data.
in	*fifo_data_size*	Size of FIFO data.
out	*p_chan_data*	Pointer to buffer where the sub-sample data can be saved.
in,out	*p_chan_data_num*	Input: Number of provided p_chan_data elements; Output: Number of used p_chan_data elements.
in,out	*p_metadata*	Structure will be used to get last sequencer information and save newest one. Furthermore, the FIFO mapping must be configured here.

Return values

ERR_SUCCESS	Function returns without error.
ERR_ARGUMENT	At least one parameter is wrong.
ERR_POINTER	Detected NULL pointer for data.

4.8 Sub-sample extraction with SAR support

Module for extracting all samples in the FIFO data stream including their SAR photodiode offset currents.

Data Structures

- struct as7058_extract2_sample_t
- struct as7058_extract2_state_t

Macros

- #define AS7058_EXTRACT2_FUNC_DECL

Functions

- AS7058_EXTRACT2_FUNC_DECL err_code_t as7058_extract2_prepare_for_new_measurement (uint32_t fifo_map, uint32_t sar_map, as7058_sar_transfer_mode_t sar_transfer_mode, as7058_extract2_state_t *p_state)

  Prepares the as7058_extract2_state_t instance for a new measurement.
- AS7058_EXTRACT2_FUNC_DECL err_code_t as7058_extract2 (const uint8_t *p_fifo_data, uint16_t fifo_data_size, as7058_extract2_sub_sample_samples_t *pp_sub_sample_samples, as7058_extract2_sample_t *p_sample_buffer, uint16_t sample_buffer_length, as7058_extract2_state_t *p_state)

  Extracts all samples from the provided FIFO data stream.

Variables

- const typedef as7058_extract2_sample_t * as7058_extract2_sub_sample_samples_t [AS7058_SUB_SAMPLE_ID_NUM+1]

4.8.1 Detailed Description

Module for extracting all samples in the FIFO data stream including their SAR photodiode offset currents.

4.8.2 Macro Definition Documentation

4.8.2.1 AS7058_EXTRACT2_FUNC_DECL `#define AS7058_EXTRACT2_FUNC_DECL`

Preprocessor macro that is placed in front of every internal function declaration with external linkage. By default this macro is defined to expand to nothing, but it can be defined externally. Example use cases include setting keywords.

4.8.3 Function Documentation

4.8.3.1 as7058_extract2_prepare_for_new_measurement() AS7058_EXTRACT2_FUNC_DECL err_code_t

```
as7058_extract2_prepare_for_new_measurement (
            uint32_t fifo_map,
            uint32_t sar_map,
            as7058_sar_transfer_mode_t sar_transfer_mode,
            as7058_extract2_state_t * p_state )
```

Prepares the as7058_extract2_state_t instance for a new measurement.

During an ongoing measurement, all received FIFO data streams must be extracted in the order they have been received using the same instance of as7058_extract2_state_t.

Parameters

in	*fifo_map*	Bitmask representing the sub-samples that are enabled in the AS7058 AFE for the current measurement, see as7058_sub_sample_flags.
in	*sar_map*	Bitmask representing the sub-samples for which SAR is enabled in the AS7058 AFE, see as7058_sub_sample_flags. In the AS7058 AFE, SAR can be enabled per PPG sub-sample but not per modulator, i.e. it is not possible to enable SAR for sub-sample 1 on PPG modulator 1 but disable it for sub-sample 1 on PPG modulator 2. Despite this, this bitmask uses the standard as7058_sub_sample_flags where there are separate bits for PPG modulator 1 and PPG modulator 2 for each sub-sample. If SAR is enabled for a sub-sample, the bits corresponding to this sub-sample on both PPG modulator 1 and PPG modulator 2 need to be set. However, bits that are not set in the FIFO map do not necessarily need to be set in the SAR map. For example, if SAR is enabled for PPG sub-sample 1 and AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 and AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 are set in the FIFO map, AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 and AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 also need to be set in the SAR map. If either AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 or AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 are not set in the FIFO map, the corresponding bit can also be left unset in the SAR map.
in	*sar_transfer_mode*	Indicates how the AS7058 AFE is configured to provide the SAR status information.
out	*p_state*	Pointer to the as7058_extract2_state_t instance to prepare.

Return values

ERR_SUCCESS	Function returns without error.
ERR_POINTER	Invalid pointer argument.
ERR_ARGUMENT	Invalid FIFO map, invalid SAR map, or invalid SAR transfer mode.

4.8.3.2 as7058_extract2() AS7058_EXTRACT2_FUNC_DECL err_code_t as7058_extract2 (

```
const uint8_t * p_fifo_data,
uint16_t fifo_data_size,
as7058_extract2_sub_sample_samples_t * pp_sub_sample_samples,
as7058_extract2_sample_t * p_sample_buffer,
uint16_t sample_buffer_length,
as7058_extract2_state_t * p_state )
```

Extracts all samples from the provided FIFO data stream.

During an ongoing measurement, all received FIFO data streams must be extracted in the order they have been received using the same instance of as7058_extract2_state_t. When starting a measurement, first call as7058_extract2_prepare_for_new_measurement to prepare the as7058_extract2_state_t instance.

Parameters

in	*p_fifo_data*	Pointer to the start of the FIFO data stream to extract.
in	*fifo_data_size*	Size of the FIFO data stream in bytes.
out	*pp_sub_sample_samples*	Pointer to an array with AS7058_SUB_SAMPLE_ID_NUM + 1 elements where each item (excluding the last one) points to the first extracted sample of the corresponding sub-sample. The extracted samples are sorted in the order they occurred in the FIFO data stream. The subsequent array element always points past the last extracted sample of the sub-sample. Please refer to the drawing in the header file for more information.
out	*p_sample_buffer*	Pointer to the buffer storing the extracted samples.
out	*sample_buffer_length*	Number of elements that can be stored in the extracted samples buffer.
in,out	*p_state*	Instance of the as7058_extract2_state_t instance.

Return values

ERR_SUCCESS	Function returns without error.
ERR_POINTER	Invalid pointer argument.
ERR_SIZE	Invalid FIFO data stream size or extracted samples buffer length.
ERR_DATA	Inconsistency detected in FIFO data stream or between configuration in as7058_extract2_state_t instance and actual configuration of AS7058 AFE.
ERR_ARGUMENT	Invalid data in the as7058_extract2_state_t instance.

4.8.4 Variable Documentation

4.8.4.1 as7058_extract2_sub_sample_samples_t const typedef as7058_extract2_sample_t* as7058_↩ extract2_sub_sample_samples_t[AS7058_SUB_SAMPLE_ID_NUM+1]

Type for an array of pointers with AS7058_SUB_SAMPLE_ID_NUM + 1 elements. The extracted samples of type as7058_extract2_sample_t are stored in a separate buffer. Inside the separate buffer, the extracted samples

of a single sub-sample are stored in contiguous memory. The first `AS7058_SUB_SAMPLE_ID_NUM` elements of this array point to the first extracted sample of the corresponding sub-sample. The subsequent array element always points past the last extracted sample of the sub-sample. See the description of as7058_extract2 for more information.

5 Data Structure Documentation

5.1 agc_configuration_t Struct Reference

Data Fields

- agc_mode_t mode
- agc_ampl_cntl_mode_t led_control_mode
- agc_channel_id_t channel
- uint8_t led_current_min
- uint8_t led_current_max
- uint8_t rel_amplitude_min_x100
- uint8_t rel_amplitude_max_x100
- uint8_t rel_amplitude_motion_x100
- uint8_t num_led_steps
- uint8_t reserved [3]
- int32_t threshold_min
- int32_t threshold_max

5.1.1 Detailed Description

Configuration of an instance of the AGC algorithm.

Note

For mode AGC_MODE_STATUS all the parameters but channel are unused and should be set to zero.

5.1.2 Field Documentation

5.1.2.1 mode `agc_mode_t agc_configuration_t::mode`

Selects the AGC algorithm mode.

5.1.2.2 led_control_mode `agc_ampl_cntl_mode_t agc_configuration_t::led_control_mode`

Selects the LED amplitude control mode.

5.1.2.3 channel `agc_channel_id_t agc_configuration_t::channel`

Selects the PPG channel that is controlled.

5.1.2.4 led_current_min `uint8_t agc_configuration_t::led_current_min`

Lower bound of the LED current range as a register value.

5.1.2.5 led_current_max `uint8_t agc_configuration_t::led_current_max`

Upper bound of the LED current range as a register value.

5.1.2.6 rel_amplitude_min_x100 `uint8_t agc_configuration_t::rel_amplitude_min_x100`

Lower bound of the targeted PPG signal amplitude, relative to the size of the target PGG signal range. The unit of this field is percent. The minimum valid value is 0, the maximum valid value is 100.

5.1.2.7 rel_amplitude_max_x100 `uint8_t agc_configuration_t::rel_amplitude_max_x100`

Upper bound of the targeted PPG signal amplitude, relative to the size of the target PGG signal range. The unit of this field is percent. This value must not be less than rel_amplitude_min_x100. The maximum valid value is 100.

5.1.2.8 rel_amplitude_motion_x100 `uint8_t agc_configuration_t::rel_amplitude_motion_x100`

Minimum PPG signal amplitude at which the PPG signal is considered a motion artifact. The threshold is relative to the size of the target PGG signal range. The unit of this field is percent. This value must not be less than rel_amplitude_max_x100. The maximum valid value is 100.

5.1.2.9 num_led_steps `uint8_t agc_configuration_t::num_led_steps`

Number of steps the LED current range is partitioned in. When the AGC algorithm determines that the LED current needs to be increased or decreased and the bounds of the LED current range are not yet reached, the LED current is adjusted by one step. If led_control_mode is set to AGC_AMPL_CNTL_MODE_AUTO, this value must not be zero and must not be greater than the size of the LED current range.

5.1.2.10 reserved `uint8_t agc_configuration_t::reserved[3]`

Padding bytes. Needs to be set to zero.

5.1.2.11 threshold_min `int32_t agc_configuration_t::threshold_min`

Lower bound of the target PPG signal range. The unit of this field is ADC counts.

5.1.2.12 threshold_max `int32_t agc_configuration_t::threshold_max`

Upper bound of the target PPG signal range. The unit of this field is ADC counts. This value must be greater than threshold_min.

5.2 agc_status_t Struct Reference

Data Fields

- agc_change_state_t pd_offset_change
- uint8_t pd_offset_current
- agc_change_state_t led_change
- uint8_t led_current

5.2.1 Detailed Description

Status information of an instance of the AGC algorithm.

5.2.2 Field Documentation

5.2.2.1 pd_offset_change `agc_change_state_t agc_status_t::pd_offset_change`

Change information for the photodiode offset current.

5.2.2.2 pd_offset_current `uint8_t agc_status_t::pd_offset_current`

Current value of the photodiode offset current as a register value.

5.2.2.3 led_change `agc_change_state_t agc_status_t::led_change`

Change information for the LED current.

5.2.2.4 led_current `uint8_t agc_status_t::led_current`

Current value of the LED current as a register value.

5.3 agc_version_t Struct Reference

Data Fields

- uint8_t major
- uint8_t minor
- uint8_t patch

5.3.1 Detailed Description

Version information of the AGC module.

5.3.2 Field Documentation

5.3.2.1 major `uint8_t agc_version_t::major`

Major version.

5.3.2.2 minor `uint8_t agc_version_t::minor`

Minor version.

5.3.2.3 patch `uint8_t agc_version_t::patch`

Patch version.

5.4 as7058_bioz_meas_config_t Struct Reference

Data Fields

- uint16_t num_averages
- uint16_t num_dropped_first_samples

5.4.1 Detailed Description

Configuration for the BioZ measurement mode.

5.4.2 Field Documentation

5.4.2.1 num_averages `uint16_t as7058_bioz_meas_config_t::num_averages`

Number of samples to average.

5.4.2.2 num_dropped_first_samples `uint16_t as7058_bioz_meas_config_t::num_dropped_first_samples`

Number of samples to drop before averaging. Because of settling time, BioZ results can be optimized by dropping the first samples.

5.5 as7058_bioz_meas_result_t Struct Reference

Data Fields

- uint32_t ref_resistor
- uint32_t ref_temperature_adc
- uint32_t temperature_adc
- int32_t slope
- struct as7058_bioz_measurement measurements [AS7058_BIOZ_MEASUREMENT_TYPE_NUM]

5.5.1 Detailed Description

Result of the BioZ measurement mode.

5.5.2 Field Documentation

5.5.2.1 ref_resistor `uint32_t as7058_bioz_meas_result_t::ref_resistor`

Resistance of the reference resistor in ohm. This value is calculated from OTP data.

5.5.2.2 ref_temperature_adc `uint32_t as7058_bioz_meas_result_t::ref_temperature_adc`

ADC counts of the temperature when the resistance of the reference resistor was measured. This value is calculated from OTP data.

5.5.2.3 temperature_adc `uint32_t as7058_bioz_meas_result_t::temperature_adc`

ADC counts of the temperature during the BioZ measurements. It is set to UINT32_MAX if temperature measurement is disabled.

5.5.2.4 slope `int32_t as7058_bioz_meas_result_t::slope`

Slope of the temperature scaled by 1000000.

5.5.2.5 measurements `struct as7058_bioz_measurement as7058_bioz_meas_result_t::measurements[AS7058_BIOZ_MEAS`

BioZ measurement results acquired with different measurement setups. See as7058_bioz_measurement_types.

5.6 as7058_bioz_measurement Struct Reference

Data Fields

- uint32_t in_phase
- uint32_t quadrature

5.6.1 Detailed Description

Result of a single BioZ measurement.

5.6.2 Field Documentation

5.6.2.1 in_phase `uint32_t as7058_bioz_measurement::in_phase`

In-phase component of the measured BioZ signal.

5.6.2.2 quadrature `uint32_t as7058_bioz_measurement::quadrature`

Quadrature component of the measured BioZ signal.

5.7 as7058_eda_scaling_config_t Struct Reference

Data Fields

- uint16_t num_averages

5.7.1 Detailed Description

Configuration for the EDA impedance scaling measurement mode.

5.7.2 Field Documentation

5.7.2.1 num_averages `uint16_t as7058_eda_scaling_config_t::num_averages`

Number of samples to average.

5.8 as7058_eda_scaling_result_t Struct Reference

Data Fields

- uint32_t ref_resistor
- uint32_t ref_temperature_adc
- uint32_t temperature_adc
- int32_t slope
- uint32_t short_p
- uint32_t short_n
- uint32_t resistor_p
- uint32_t resistor_n

5.8.1 Detailed Description

Result of the EDA impedance scaling measurement mode.

5.8.2 Field Documentation

5.8.2.1 ref_resistor `uint32_t as7058_eda_scaling_result_t::ref_resistor`

Resistance of the reference resistor in ohm. This value is calculated from OTP data.

5.8.2.2 ref_temperature_adc `uint32_t as7058_eda_scaling_result_t::ref_temperature_adc`

ADC counts of the temperature when the resistance of the reference resistor was measured. This value is calculated from OTP data.

5.8.2.3 temperature_adc `uint32_t as7058_eda_scaling_result_t::temperature_adc`

ADC counts of the temperature during the EDA scaling measurements. It is set to UINT32_MAX if temperature measurement is disabled.

5.8.2.4 slope `int32_t as7058_eda_scaling_result_t::slope`

Slope of the temperature scaled by 1000000.

5.8.2.5 short_p `uint32_t as7058_eda_scaling_result_t::short_p`

Measured EDA+ while electrical circuit has a short.

5.8.2.6 short_n `uint32_t as7058_eda_scaling_result_t::short_n`

Measured EDA- while electrical circuit has a short.

5.8.2.7 resistor_p `uint32_t as7058_eda_scaling_result_t::resistor_p`

Measured EDA+ while electrical circuit is connected with the reference resistor.

5.8.2.8 resistor_n `uint32_t as7058_eda_scaling_result_t::resistor_n`

Measured EDA- while electrical circuit is connected with the reference resistor.

5.9 as7058_extract2_sample_t Struct Reference

Data Fields

- uint32_t pd_offset: 8
- uint32_t sar_enabled: 1
- uint32_t reserved: 3
- uint32_t adc_value: 20

5.9.1 Detailed Description

Sample extracted from the FIFO buffer.

5.9.2 Field Documentation

5.9.2.1 pd_offset `uint32_t as7058_extract2_sample_t::pd_offset`

SAR-controlled value of the photodiode offset current. If SAR is enabled, this field contains the current photodiode offset current. SAR controls the upper four bits of the photodiode offset current. The lower four bits can be set to a constant value by writing to a register. In SAR transfer mode AS7058_SAR_TRANSFER_MODE_SEPARATE, the full photodiode offset current is provided. In SAR transfer mode AS7058_SAR_TRANSFER_MODE_INLINE, only the SAR-controlled bits are provided and the lower four bits are always set to zero. The full photodiode offset current can be obtained by combining the value of this field with the lower four bits of the value written to the register using a bitwise OR operation.

5.9.2.2 sar_enabled `uint32_t as7058_extract2_sample_t::sar_enabled`

SAR is enabled if this field is non-zero.

5.9.2.3 reserved `uint32_t as7058_extract2_sample_t::reserved`

Reserved for future use.

5.9.2.4 adc_value `uint32_t as7058_extract2_sample_t::adc_value`

ADC value of the acquired sample.

5.10 as7058_extract2_state_t Struct Reference

Data Fields

- uint32_t fifo_map
- uint32_t sar_map
- uint8_t reserved
- as7058_sar_transfer_mode_t sar_transfer_mode
- as7058_modulator_sub_sample_t last_ppg_sub_samples [AS7058_MODULATOR_NUM_PPG]
- uint8_t last_pd_offsets [AS7058_MODULATOR_NUM_PPG][AS7058_MODULATOR_SUB_SAMPLE_NUM_PPG]
- uint32_t last_pd_offsets_valid

5.10.1 Detailed Description

State of the sample extraction. During an ongoing measurement, all received FIFO data streams must be extracted in the order they have been received using the same instance of this structure. Do not modify the members of this structure directly.

5.10.2 Field Documentation

5.10.2.1 fifo_map `uint32_t as7058_extract2_state_t::fifo_map`

Bitmask representing the sub-samples that are enabled in the AS7058 AFE for the current measurement, see as7058_sub_sample_flags.

5.10.2.2 sar_map `uint32_t as7058_extract2_state_t::sar_map`

Bitmask representing the sub-samples for which SAR is enabled in the AS7058 AFE, see as7058_sub_sample_flags. In the AS7058 AFE, SAR can be enabled per PPG sub-sample but not per modulator, i.e. it is not possible to enable SAR for sub-sample 1 on PPG modulator 1 but disable it for sub-sample 1 on PPG modulator 2. Despite this, this bitmask uses the standard as7058_sub_sample_flags where there are separate bits for PPG modulator 1 and PPG modulator 2 for each sub-sample. If SAR is enabled for a sub-sample, the bits corresponding to this sub-sample on both PPG modulator 1 and PPG modulator 2 need to be set. However, bits that are not set in the FIFO map do not necessarily need to be set in the SAR map. For example, if SAR is enabled for PPG sub-sample 1 and AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 and AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 are set in the FIFO map, AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 and AS7058_SUB_SAMPLE_FLAG_PPG2_SUB1 also need to be set in the SAR map. If either AS7058_SUB_SAMPLE_FLAG_PPG1_SUB1 or AS7058_SUB_SAMPLE_FLAG_PPG2_SUB are not set in the FIFO map, the corresponding bit can also be left unset in the SAR map.

5.10.2.3 reserved `uint8_t as7058_extract2_state_t::reserved`

Reserved for future use.

5.10.2.4 sar_transfer_mode `as7058_sar_transfer_mode_t as7058_extract2_state_t::sar_transfer_mode`

Indicates how the AS7058 AFE is configured to provide the SAR status information.

5.10.2.5 last_ppg_sub_samples `as7058_modulator_sub_sample_t as7058_extract2_state_t::last_ppg_sub_samples[AS7058_MODULATOR_NUM_PPG]`

Contains to which sub-sample the last received sample of a PPG modulator belongs.

5.10.2.6 last_pd_offsets `uint8_t as7058_extract2_state_t::last_pd_offsets[AS7058_MODULATOR_NUM_PPG][AS7058_MODU`

Contains the last received SAR photodiode offset current per sub-sample for each PPG modulator.

5.10.2.7 last_pd_offsets_valid `uint32_t as7058_extract2_state_t::last_pd_offsets_valid`

Bitmask representing the sub-samples for which last_pd_offsets contains a valid value, see as7058_sub_sample_flags.

5.11 as7058_extract_metadata_t Struct Reference

Data Fields

- uint32_t fifo_map
- struct extract_metadata current
- struct extract_metadata recent
- uint8_t copy_recent_to_current

5.11.1 Detailed Description

Metadata for extraction function.

5.11.2 Field Documentation

5.11.2.1 fifo_map `uint32_t as7058_extract_metadata_t::fifo_map`

Currently available sub-samples inside the FIFO data stream. See as7058_sub_sample_flags and as7058_meas_config_t.

5.11.2.2 current `struct extract_metadata as7058_extract_metadata_t::current`

Current metadata.

5.11.2.3 recent `struct extract_metadata as7058_extract_metadata_t::recent`

Recent metadata.

5.11.2.4 copy_recent_to_current `uint8_t as7058_extract_metadata_t::copy_recent_to_current`

Non-zero value: Current meta data will be overwritten by recent metadata.

5.12 as7058_meas_config_t Struct Reference

Data Fields

- uint32_t ppg_sample_period_us
- uint32_t ecg_seq1_sample_period_us
- uint32_t ecg_seq2_sample_period_us
- uint32_t fifo_map
- as7058_sub_sample_ids_t agc_channels [AGC_MAX_CHANNEL_CNT]
- uint32_t sar_map
- as7058_sar_transfer_mode_t sar_transfer_mode
- uint8_t reserved [3]

5.12.1 Detailed Description

Measurement settings which can be read out after finished register configuration.

5.12.2 Field Documentation

5.12.2.1 ppg_sample_period_us `uint32_t as7058_meas_config_t::ppg_sample_period_us`

Sample period for PPG in microseconds.

5.12.2.2 ecg_seq1_sample_period_us `uint32_t as7058_meas_config_t::ecg_seq1_sample_period_us`

Sample period for ECG sequence 1 in microseconds.

5.12.2.3 ecg_seq2_sample_period_us `uint32_t as7058_meas_config_t::ecg_seq2_sample_period_us`

Sample period for ECG sequence 2 in microseconds.

5.12.2.4 fifo_map `uint32_t as7058_meas_config_t::fifo_map`

Definition which sub-samples are mapped inside FIFO. More than one flag can be set. See as7058_sub_sample_flags.

5.12.2.5 agc_channels `as7058_sub_sample_ids_t as7058_meas_config_t::agc_channels[AGC_MAX_CHANNEL_CNT]`

Activated AGC channels. AS7058_SUB_SAMPLE_ID_DISABLED is set on unused channels.

5.12.2.6 sar_map `uint32_t as7058_meas_config_t::sar_map`

Defines which sub-samples have SAR enabled. More than one flag can be set. See as7058_sub_sample_flags.

5.12.2.7 sar_transfer_mode `as7058_sar_transfer_mode_t as7058_meas_config_t::sar_transfer_mode`

Defines how the SAR status is provided by the AS7058 AFE.

5.12.2.8 reserved `uint8_t as7058_meas_config_t::reserved[3]`

Padding bytes, reserved for future use.

5.13 as7058_pd_offset_calibration_config_t Struct Reference

Data Fields

- as7058_sub_sample_ids_t sub_sample_id
- uint8_t reserved
- uint16_t num_averages
- uint16_t ppg_sample_period_multiplier

5.13.1 Detailed Description

Measurement configuration for PD offset calibration.

5.13.2 Field Documentation

5.13.2.1 sub_sample_id `as7058_sub_sample_ids_t as7058_pd_offset_calibration_config_t::sub_sample_id`

Sub-sample to use for calibration, see as7058_sub_sample_ids. Depending on the selected sub-sample, either PPG modulator 1 or PPG modulator 2 is calibrated. ECG sub-samples must not be selected.

5.13.2.2 reserved `uint8_t as7058_pd_offset_calibration_config_t::reserved`

Padding byte, reserved for future use. Must be set to zero.

5.13.2.3 num_averages `uint16_t as7058_pd_offset_calibration_config_t::num_averages`

Number of samples to acquire and average per calibration point measurement.

5.13.2.4 ppg_sample_period_multiplier `uint16_t as7058_pd_offset_calibration_config_t::ppg_sample_period_multiplier`

Base sample period multiplier to use during calibration. The base sample period is 31.25 microseconds. The resulting sample period is obtained by incrementing the multiplier by one and by multiplying the base sample period by the incremented multiplier. For example, a multiplier value of 9 results in sample period of (9 + 1) ∗ 31.25 microseconds = 312.5 microseconds. The contents of registers AS7058_REGADDR_PPG_FREQL and AS7058_REGADDR_PPG_FREQH are temporarily replaced by the provided value during calibration.

5.14 as7058_pd_offset_calibration_result_t Struct Reference

Data Fields

- uint32_t compensation_table [AS7058_PD_OFFSET_CALIBRATION_COMPENSATION_TABLE_ELEMENTS_NUM]

5.14.1 Detailed Description

Result structure for PD offset calibration.

5.14.2 Field Documentation

5.14.2.1 compensation_table `uint32_t as7058_pd_offset_calibration_result_t::compensation_table[AS7058_PD_OFFSET_CALIBRATION_COMPENSATION_TABLE_ELEMENTS_NUM]`

Contains the measured ADC count offsets for different photodiode offset current values. PD offset calibration takes the lower 4 bits of the photodiode offset current of the selected PPG1 or PPG2 sub-sample from the chip configuration and modifies the upper 4 bits during calibration. The first item in the array contains the ADC count offset when the upper 4 bits are set to value 1. The last item contains the offset when the upper 4 bits are set to value 15.

5.15 as7058_reg_group_control_t Union Reference

Data Structures

- struct control_regs

Data Fields

- struct as7058_reg_group_control_t::control_regs reg_vals
- uint8_t reg_buffer [sizeof(struct control_regs)]

5.15.1 Detailed Description

Register group for control configuration.

5.15.2 Field Documentation

5.15.2.1 reg_vals `struct as7058_reg_group_control_t::control_regs as7058_reg_group_control_t::reg_vals`

Register values for control group.

5.15.2.2 reg_buffer `uint8_t as7058_reg_group_control_t::reg_buffer[sizeof(struct control_regs)]`

Register buffer for control configuration.

5.16 as7058_reg_group_ecg_t Union Reference

Data Structures

- struct ecg_regs

Data Fields

- struct as7058_reg_group_ecg_t::ecg_regs reg_vals
- uint8_t reg_buffer [sizeof(struct ecg_regs)]

5.16.1 Detailed Description

Register group for configuration of ECG.

5.16.2 Field Documentation

5.16.2.1 reg_vals `struct as7058_reg_group_ecg_t::ecg_regs as7058_reg_group_ecg_t::reg_vals`

Register values for ECG group.

5.16.2.2 reg_buffer `uint8_t as7058_reg_group_ecg_t::reg_buffer[sizeof(struct ecg_regs)]`

Register buffer for ECG configuration.

5.17 as7058_reg_group_fifo_t Union Reference

Data Structures

- struct fifo_regs

Data Fields

- struct as7058_reg_group_fifo_t::fifo_regs reg_vals
- uint8_t reg_buffer [sizeof(struct fifo_regs)]

5.17.1 Detailed Description

Register group for configuration of the FIFO handling.

5.17.2 Field Documentation

5.17.2.1 reg_vals `struct as7058_reg_group_fifo_t::fifo_regs as7058_reg_group_fifo_t::reg_vals`

Register values for FIFO group.

5.17.2.2 reg_buffer `uint8_t as7058_reg_group_fifo_t::reg_buffer[sizeof(struct fifo_regs)]`

Register buffer for FIFO configuration.

5.18 as7058_reg_group_iir_t Union Reference

Data Structures

- struct iir_regs

Data Fields

- struct as7058_reg_group_iir_t::iir_regs reg_vals
- uint8_t reg_buffer [sizeof(struct iir_regs)]

5.18.1 Detailed Description

Register group for configuration of the Infinite Impulse Response filter (IIR filter).

5.18.2 Field Documentation

5.18.2.1 reg_vals `struct as7058_reg_group_iir_t::iir_regs as7058_reg_group_iir_t::reg_vals`

Register values for IIR group.

5.18.2.2 reg_buffer `uint8_t as7058_reg_group_iir_t::reg_buffer[sizeof(struct iir_regs)]`

Register buffer for IIR filter configuration.

5.19 as7058_reg_group_ios_t Union Reference

Data Structures

- struct ios_regs

Data Fields

- struct as7058_reg_group_ios_t::ios_regs reg_vals
- uint8_t reg_buffer [sizeof(struct ios_regs)]

5.19.1 Detailed Description

Register group for configuration of the PD offset currents.

5.19.2 Field Documentation

5.19.2.1 reg_vals `struct as7058_reg_group_ios_t::ios_regs as7058_reg_group_ios_t::reg_vals`

Register values for IOS group.

5.19.2.2 reg_buffer `uint8_t as7058_reg_group_ios_t::reg_buffer[sizeof(struct ios_regs)]`

Register buffer for PD offset current configuration.

5.20 as7058_reg_group_led_t Union Reference

Data Structures

- struct led_regs

Data Fields

- struct as7058_reg_group_led_t::led_regs reg_vals
- uint8_t reg_buffer [sizeof(struct led_regs)]

5.20.1 Detailed Description

Register group for configuration of the LEDs.

5.20.2 Field Documentation

5.20.2.1 reg_vals `struct as7058_reg_group_led_t::led_regs as7058_reg_group_led_t::reg_vals`

Register values for LED group.

5.20.2.2 reg_buffer `uint8_t as7058_reg_group_led_t::reg_buffer[sizeof(struct led_regs)]`

Register buffer for LED configuration.

5.21 as7058_reg_group_pd_t Union Reference

Data Structures

- struct pd_regs

Data Fields

- struct as7058_reg_group_pd_t::pd_regs reg_vals
- uint8_t reg_buffer [sizeof(struct pd_regs)]

5.21.1 Detailed Description

Register group for configuration of the photodiodes.

5.21.2 Field Documentation

5.21.2.1 reg_vals `struct as7058_reg_group_pd_t::pd_regs as7058_reg_group_pd_t::reg_vals`

Register values for PD group.

5.21.2.2 reg_buffer `uint8_t as7058_reg_group_pd_t::reg_buffer[sizeof(struct pd_regs)]`

Register buffer for photodiodes configuration.

5.22 as7058_reg_group_power_t Union Reference

Data Structures

- struct power_regs

Data Fields

- struct as7058_reg_group_power_t::power_regs reg_vals
- uint8_t reg_buffer [sizeof(struct power_regs)]

5.22.1 Detailed Description

Register group for power configuration.

Note

Registers PWR_ON and PWR_ISO were moved to the beginning because those registers need to be set first to access the other one. Furthermore, those registers are not in the same address range.

5.22.2 Field Documentation

5.22.2.1 reg_vals `struct as7058_reg_group_power_t::power_regs as7058_reg_group_power_t::reg_vals`

Register values for power group.

5.22.2.2 reg_buffer `uint8_t as7058_reg_group_power_t::reg_buffer[sizeof(struct power_regs)]`

Register buffer for power configuration.

5.23 as7058_reg_group_pp_t Union Reference

Data Structures

- struct pp_regs

Data Fields

- struct as7058_reg_group_pp_t::pp_regs reg_vals
- uint8_t reg_buffer [sizeof(struct pp_regs)]

5.23.1 Detailed Description

Register group for configuration of post-processing engine.

5.23.2 Field Documentation

5.23.2.1 reg_vals `struct as7058_reg_group_pp_t::pp_regs as7058_reg_group_pp_t::reg_vals`

Register values for post-processing group.

5.23.2.2 reg_buffer `uint8_t as7058_reg_group_pp_t::reg_buffer[sizeof(struct pp_regs)]`

Register buffer for post-processing configuration.

5.24 as7058_reg_group_ppg_t Union Reference

Data Structures

- struct ppg_regs

Data Fields

- struct as7058_reg_group_ppg_t::ppg_regs reg_vals
- uint8_t reg_buffer [sizeof(struct ppg_regs)]

5.24.1 Detailed Description

Register group for configuration of PPG.

5.24.2 Field Documentation

5.24.2.1 reg_vals `struct as7058_reg_group_ppg_t::ppg_regs as7058_reg_group_ppg_t::reg_vals`

Register values for PPG group.

5.24.2.2 reg_buffer `uint8_t as7058_reg_group_ppg_t::reg_buffer[sizeof(struct ppg_regs)]`

Register buffer for PPG configuration.

5.25 as7058_reg_group_seq_t Union Reference

Data Structures

- struct seq_regs

Data Fields

- struct as7058_reg_group_seq_t::seq_regs reg_vals
- uint8_t reg_buffer [sizeof(struct seq_regs)]

5.25.1 Detailed Description

Register group for configuration of the sequencer.

5.25.2 Field Documentation

5.25.2.1 reg_vals `struct as7058_reg_group_seq_t::seq_regs as7058_reg_group_seq_t::reg_vals`

Register values for sequencer group.

5.25.2.2 reg_buffer `uint8_t as7058_reg_group_seq_t::reg_buffer[sizeof(struct seq_regs)]`

Register buffer for sequencer configuration.

5.26 as7058_reg_group_sinc_t Union Reference

Data Structures

- struct sinc_regs

Data Fields

- struct as7058_reg_group_sinc_t::sinc_regs reg_vals
- uint8_t reg_buffer [sizeof(struct sinc_regs)]

5.26.1 Detailed Description

Register group for configuration of the sinc filter.

5.26.2 Field Documentation

5.26.2.1 reg_vals `struct as7058_reg_group_sinc_t::sinc_regs as7058_reg_group_sinc_t::reg_vals`

Register values for SINC group.

5.26.2.2 reg_buffer `uint8_t as7058_reg_group_sinc_t::reg_buffer[sizeof(struct sinc_regs)]`

Register buffer for SINC filter configuration.

5.27 as7058_special_measurement_result_t Union Reference

Data Fields

- as7058_bioz_meas_result_t bioz
- as7058_eda_scaling_result_t eda_scaling
- as7058_pd_offset_calibration_result_t pd_offset_calibration

5.27.1 Detailed Description

Special measurement results.

5.27.2 Field Documentation

5.27.2.1 bioz `as7058_bioz_meas_result_t as7058_special_measurement_result_t::bioz`

Result for mode AS7058_MEAS_MODE_SPECIAL_BIOZ.

5.27.2.2 eda_scaling `as7058_eda_scaling_result_t as7058_special_measurement_result_t::eda_↩ scaling`

Result for mode AS7058_MEAS_MODE_SPECIAL_SCALING_EDA.

5.27.2.3 pd_offset_calibration `as7058_pd_offset_calibration_result_t as7058_special_measurement_result_t::pd_offset_calibration`

Result for mode AS7058_MEAS_MODE_SPECIAL_PD_OFFSET_CALIBRATION.

5.28 as7058_reg_group_control_t::control_regs Struct Reference

Data Fields

- uint8_t i2c_mode
- uint8_t int_cfg
- uint8_t if_cfg
- uint8_t gpio_cfg1
- uint8_t gpio_cfg2
- uint8_t io_cfg

5.28.1 Detailed Description

Contents of the Control register group.

5.28.2 Field Documentation

5.28.2.1 i2c_mode `uint8_t as7058_reg_group_control_t::control_regs::i2c_mode`

Content of register I2C_MODE.

5.28.2.2 int_cfg `uint8_t as7058_reg_group_control_t::control_regs::int_cfg`

Content of register INT_CFG.

5.28.2.3 if_cfg `uint8_t as7058_reg_group_control_t::control_regs::if_cfg`

Content of register IF_CFG.

5.28.2.4 gpio_cfg1 `uint8_t as7058_reg_group_control_t::control_regs::gpio_cfg1`

Content of register GPIO_CFG1.

5.28.2.5 gpio_cfg2 `uint8_t as7058_reg_group_control_t::control_regs::gpio_cfg2`

Content of register GPIO_CFG2.

5.28.2.6 io_cfg `uint8_t as7058_reg_group_control_t::control_regs::io_cfg`

Content of register IO_CFG.

5.29 as7058_reg_group_ecg_t::ecg_regs Struct Reference

Data Fields

- uint8_t bioz_cfg
- uint8_t bioz_excit
- uint8_t bioz_mixer
- uint8_t bioz_select
- uint8_t bioz_gain
- uint8_t ecgmod_cfg1
- uint8_t ecgmod_cfg2
- uint8_t ecgimux_cfg1
- uint8_t ecgimux_cfg2
- uint8_t ecgimux_cfg3
- uint8_t ecgamp_cfg1
- uint8_t ecgamp_cfg2
- uint8_t ecgamp_cfg3
- uint8_t ecgamp_cfg4
- uint8_t ecgamp_cfg5
- uint8_t ecgamp_cfg6
- uint8_t ecgamp_cfg7
- uint8_t ecg_bioz
- uint8_t leadoff_cfg
- uint8_t leadoff_thresl
- uint8_t leadoff_thresh

5.29.1 Detailed Description

Contents of the ECG register group.

5.29.2 Field Documentation

5.29.2.1 bioz_cfg `uint8_t as7058_reg_group_ecg_t::ecg_regs::bioz_cfg`

Content of register BIOZ_CFG.

5.29.2.2 bioz_excit `uint8_t as7058_reg_group_ecg_t::ecg_regs::bioz_excit`

Content of register BIOZ_EXCIT.

5.29.2.3 bioz_mixer `uint8_t as7058_reg_group_ecg_t::ecg_regs::bioz_mixer`

Content of register BIOZ_MIXER.

5.29.2.4 bioz_select `uint8_t as7058_reg_group_ecg_t::ecg_regs::bioz_select`

Content of register BIOZ_SELECT.

5.29.2.5 bioz_gain `uint8_t as7058_reg_group_ecg_t::ecg_regs::bioz_gain`

Content of register BIOZ_GAIN.

5.29.2.6 ecgmod_cfg1 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgmod_cfg1`

Content of register ECGMOD_CFG1.

5.29.2.7 ecgmod_cfg2 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgmod_cfg2`

Content of register ECGMOD_CFG2.

5.29.2.8 ecgimux_cfg1 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgimux_cfg1`

Content of register ECGIMUX_CFG1.

5.29.2.9 ecgimux_cfg2 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgimux_cfg2`

Content of register ECGIMUX_CFG2.

5.29.2.10 ecgimux_cfg3 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgimux_cfg3`

Content of register ECGIMUX_CFG3.

5.29.2.11 ecgamp_cfg1 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg1`

Content of register ECGAMP_CFG1.

5.29.2.12 ecgamp_cfg2 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg2`

Content of register ECGAMP_CFG2.

5.29.2.13 ecgamp_cfg3 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg3`

Content of register ECGAMP_CFG3.

5.29.2.14 ecgamp_cfg4 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg4`

Content of register ECGAMP_CFG4.

5.29.2.15 ecgamp_cfg5 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg5`

Content of register ECGAMP_CFG5.

5.29.2.16 ecgamp_cfg6 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg6`

Content of register ECGAMP_CFG6.

5.29.2.17 ecgamp_cfg7 `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecgamp_cfg7`

Content of register ECGAMP_CFG7.

5.29.2.18 ecg_bioz `uint8_t as7058_reg_group_ecg_t::ecg_regs::ecg_bioz`

Content of register ECG_BIOZ.

5.29.2.19 leadoff_cfg `uint8_t as7058_reg_group_ecg_t::ecg_regs::leadoff_cfg`

Content of register LEADOFF_CFG

5.29.2.20 leadoff_thresl `uint8_t as7058_reg_group_ecg_t::ecg_regs::leadoff_thresl`

Content of register LEADOFF_THRESL.

5.29.2.21 leadoff_thresh `uint8_t as7058_reg_group_ecg_t::ecg_regs::leadoff_thresh`

Content of register LEADOFF_THRESH.

5.30 extract_metadata Struct Reference

Data Fields

- as7058_sub_sample_flags_t ppg1_sub
- as7058_sub_sample_flags_t ppg2_sub

5.30.1 Detailed Description

Metadata description.

5.30.2 Field Documentation

5.30.2.1 ppg1_sub `as7058_sub_sample_flags_t extract_metadata::ppg1_sub`

Sub-sample of PPG1.

5.30.2.2 ppg2_sub `as7058_sub_sample_flags_t extract_metadata::ppg2_sub`

Sub-sample of PPG2.

5.31 as7058_reg_group_fifo_t::fifo_regs Struct Reference

Data Fields

- uint8_t fifo_threshold
- uint8_t fifo_ctrl

5.31.1 Detailed Description

Contents of the FIFO register group.

5.31.2 Field Documentation

5.31.2.1 fifo_threshold `uint8_t as7058_reg_group_fifo_t::fifo_regs::fifo_threshold`

Content of register FIFO_THRESHOLD.

5.31.2.2 fifo_ctrl `uint8_t as7058_reg_group_fifo_t::fifo_regs::fifo_ctrl`

Content of register FIFO_CTRL.

5.32 as7058_reg_group_iir_t::iir_regs Struct Reference

Data Fields

- uint8_t iir_cfg
- uint8_t reserved
- int16_t iir_coeff_data_sos [12][5]

5.32.1 Detailed Description

Contents of the IIR Filter register group.

5.32.2 Field Documentation

5.32.2.1 iir_cfg `uint8_t as7058_reg_group_iir_t::iir_regs::iir_cfg`

Content of register IIR_CFG.

5.32.2.2 reserved `uint8_t as7058_reg_group_iir_t::iir_regs::reserved`

Unused padding byte.

5.32.2.3 iir_coeff_data_sos `int16_t as7058_reg_group_iir_t::iir_regs::iir_coeff_data_sos[12][5]`

Coefficients of the cascaded IIR second-order sections. Up to 12 sections are supported. Each section has five coefficients.

5.33 as7058_reg_group_ios_t::ios_regs Struct Reference

Data Fields

- uint8_t ios_ppg1_sub1
- uint8_t ios_ppg1_sub2
- uint8_t ios_ppg1_sub3
- uint8_t ios_ppg1_sub4
- uint8_t ios_ppg1_sub5
- uint8_t ios_ppg1_sub6
- uint8_t ios_ppg1_sub7
- uint8_t ios_ppg1_sub8
- uint8_t ios_ppg2_sub1
- uint8_t ios_ppg2_sub2
- uint8_t ios_ppg2_sub3
- uint8_t ios_ppg2_sub4
- uint8_t ios_ppg2_sub5
- uint8_t ios_ppg2_sub6
- uint8_t ios_ppg2_sub7
- uint8_t ios_ppg2_sub8
- uint8_t ios_ledoff
- uint8_t ios_cfg
- uint8_t aoc_sar_thres
- uint8_t aoc_sar_range
- uint8_t aoc_sar_ppg1
- uint8_t aoc_sar_ppg2

5.33.1 Detailed Description

Contents of the PD Offset Control register group.

5.33.2 Field Documentation

5.33.2.1 ios_ppg1_sub1 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub1`

Content of register IOS_PPG1_SUB1.

5.33.2.2 ios_ppg1_sub2 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub2`

Content of register IOS_PPG1_SUB2.

5.33.2.3 ios_ppg1_sub3 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub3`

Content of register IOS_PPG1_SUB3.

5.33.2.4 ios_ppg1_sub4 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub4`

Content of register IOS_PPG1_SUB4.

5.33.2.5 ios_ppg1_sub5 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub5`

Content of register IOS_PPG1_SUB5.

5.33.2.6 ios_ppg1_sub6 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub6`

Content of register IOS_PPG1_SUB6.

5.33.2.7 ios_ppg1_sub7 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub7`

Content of register IOS_PPG1_SUB7.

5.33.2.8 ios_ppg1_sub8 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg1_sub8`

Content of register IOS_PPG1_SUB8.

5.33.2.9 ios_ppg2_sub1 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub1`

Content of register IOS_PPG2_SUB1.

5.33.2.10 ios_ppg2_sub2 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub2`

Content of register IOS_PPG2_SUB2.

5.33.2.11 ios_ppg2_sub3 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub3`

Content of register IOS_PPG2_SUB3.

5.33.2.12 ios_ppg2_sub4 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub4`

Content of register IOS_PPG2_SUB4.

5.33.2.13 ios_ppg2_sub5 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub5`

Content of register IOS_PPG2_SUB5.

5.33.2.14 ios_ppg2_sub6 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub6`

Content of register IOS_PPG2_SUB6.

5.33.2.15 ios_ppg2_sub7 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub7`

Content of register IOS_PPG2_SUB7.

5.33.2.16 ios_ppg2_sub8 `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ppg2_sub8`

Content of register IOS_PPG2_SUB8.

5.33.2.17 ios_ledoff `uint8_t as7058_reg_group_ios_t::ios_regs::ios_ledoff`

Content of register IOS_LEDOFF.

5.33.2.18 ios_cfg `uint8_t as7058_reg_group_ios_t::ios_regs::ios_cfg`

Content of register IOS_CFG.

5.33.2.19 aoc_sar_thres `uint8_t as7058_reg_group_ios_t::ios_regs::aoc_sar_thres`

Content of register AOC_SAR_THRES.

5.33.2.20 aoc_sar_range `uint8_t as7058_reg_group_ios_t::ios_regs::aoc_sar_range`

Content of register AOC_SAR_RANGE.

5.33.2.21 aoc_sar_ppg1 `uint8_t as7058_reg_group_ios_t::ios_regs::aoc_sar_ppg1`

Content of register AOC_SAR_PPG1.

5.33.2.22 aoc_sar_ppg2 `uint8_t as7058_reg_group_ios_t::ios_regs::aoc_sar_ppg2`

Content of register AOC_SAR_PPG2.

5.34 as7058_reg_group_led_t::led_regs Struct Reference

Data Fields

- uint8_t vcsel_password
- uint8_t vcsel_cfg
- uint8_t vcsel_mode
- uint8_t led_cfg
- uint8_t led_drv1
- uint8_t led_drv2
- uint8_t led1_ictrl
- uint8_t led2_ictrl
- uint8_t led3_ictrl
- uint8_t led4_ictrl
- uint8_t led5_ictrl
- uint8_t led6_ictrl
- uint8_t led7_ictrl
- uint8_t led8_ictrl
- uint8_t led_irng1
- uint8_t led_irng2
- uint8_t led_sub1
- uint8_t led_sub2
- uint8_t led_sub3
- uint8_t led_sub4
- uint8_t led_sub5
- uint8_t led_sub6
- uint8_t led_sub7
- uint8_t led_sub8
- uint8_t lowvds_wait

5.34.1 Detailed Description

Contents of the LED register group.

5.34.2 Field Documentation

5.34.2.1 vcsel_password `uint8_t as7058_reg_group_led_t::led_regs::vcsel_password`

Content of register VCSEL_PASSWORD.

5.34.2.2 vcsel_cfg `uint8_t as7058_reg_group_led_t::led_regs::vcsel_cfg`

Content of register VCSEL_CFG.

5.34.2.3 vcsel_mode `uint8_t as7058_reg_group_led_t::led_regs::vcsel_mode`

Content of register VCSEL_MODE.

5.34.2.4 led_cfg `uint8_t as7058_reg_group_led_t::led_regs::led_cfg`

Content of register LED_CFG.

5.34.2.5 led_drv1 `uint8_t as7058_reg_group_led_t::led_regs::led_drv1`

Content of register LED_DRV1.

5.34.2.6 led_drv2 `uint8_t as7058_reg_group_led_t::led_regs::led_drv2`

Content of register LED_DRV2.

5.34.2.7 led1_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led1_ictrl`

Content of register LED1_ICTRL.

5.34.2.8 led2_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led2_ictrl`

Content of register LED2_ICTRL.

5.34.2.9 led3_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led3_ictrl`

Content of register LED3_ICTRL.

5.34.2.10 led4_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led4_ictrl`

Content of register LED4_ICTRL.

5.34.2.11 led5_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led5_ictrl`

Content of register LED5_ICTRL.

5.34.2.12 led6_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led6_ictrl`

Content of register LED6_ICTRL.

5.34.2.13 led7_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led7_ictrl`

Content of register LED7_ICTRL.

5.34.2.14 led8_ictrl `uint8_t as7058_reg_group_led_t::led_regs::led8_ictrl`

Content of register LED8_ICTRL.

5.34.2.15 led_irng1 `uint8_t as7058_reg_group_led_t::led_regs::led_irng1`

Content of register LED_IRNG1.

5.34.2.16 led_irng2 `uint8_t as7058_reg_group_led_t::led_regs::led_irng2`

Content of register LED_IRNG2.

5.34.2.17 led_sub1 `uint8_t as7058_reg_group_led_t::led_regs::led_sub1`

Content of register LED_SUB1.

5.34.2.18 led_sub2 `uint8_t as7058_reg_group_led_t::led_regs::led_sub2`

Content of register LED_SUB2.

5.34.2.19 led_sub3 `uint8_t as7058_reg_group_led_t::led_regs::led_sub3`

Content of register LED_SUB3.

5.34.2.20 led_sub4 `uint8_t as7058_reg_group_led_t::led_regs::led_sub4`

Content of register LED_SUB4.

5.34.2.21 led_sub5 `uint8_t as7058_reg_group_led_t::led_regs::led_sub5`

Content of register LED_SUB5.

5.34.2.22 led_sub6 `uint8_t as7058_reg_group_led_t::led_regs::led_sub6`

Content of register LED_SUB6.

5.34.2.23 led_sub7 `uint8_t as7058_reg_group_led_t::led_regs::led_sub7`

Content of register LED_SUB7.

5.34.2.24 led_sub8 `uint8_t as7058_reg_group_led_t::led_regs::led_sub8`

Content of register LED_SUB8.

5.34.2.25 lowvds_wait `uint8_t as7058_reg_group_led_t::led_regs::lowvds_wait`

Content of register LOWVDS_WAIT.

5.35 as7058_reg_group_pd_t::pd_regs Struct Reference

Data Fields

- uint8_t pdsel_cfg
- uint8_t ppg1_pdsel1
- uint8_t ppg1_pdsel2
- uint8_t ppg1_pdsel3
- uint8_t ppg1_pdsel4
- uint8_t ppg1_pdsel5
- uint8_t ppg1_pdsel6
- uint8_t ppg1_pdsel7
- uint8_t ppg1_pdsel8
- uint8_t ppg2_pdsel1
- uint8_t ppg2_pdsel2
- uint8_t ppg2_pdsel3
- uint8_t ppg2_pdsel4
- uint8_t ppg2_pdsel5
- uint8_t ppg2_pdsel6
- uint8_t ppg2_pdsel7
- uint8_t ppg2_pdsel8
- uint8_t ppg2_afesel1
- uint8_t ppg2_afesel2
- uint8_t ppg2_afesel3
- uint8_t ppg2_afesel4
- uint8_t ppg2_afeen

5.35.1 Detailed Description

Contents of the Photodiodes register group.

5.35.2 Field Documentation

5.35.2.1 pdsel_cfg `uint8_t as7058_reg_group_pd_t::pd_regs::pdsel_cfg`

Content of register PDSEL_CFG.

5.35.2.2 ppg1_pdsel1 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel1`

Content of register PPG1_PDSEL1.

5.35.2.3 ppg1_pdsel2 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel2`

Content of register PPG1_PDSEL2.

5.35.2.4 ppg1_pdsel3 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel3`

Content of register PPG1_PDSEL3.

5.35.2.5 ppg1_pdsel4 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel4`

Content of register PPG1_PDSEL4.

5.35.2.6 ppg1_pdsel5 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel5`

Content of register PPG1_PDSEL5.

5.35.2.7 ppg1_pdsel6 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel6`

Content of register PPG1_PDSEL6.

5.35.2.8 ppg1_pdsel7 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel7`

Content of register PPG1_PDSEL7.

5.35.2.9 ppg1_pdsel8 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg1_pdsel8`

Content of register PPG1_PDSEL8.

5.35.2.10 ppg2_pdsel1 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel1`

Content of register PPG2_PDSEL1.

5.35.2.11 ppg2_pdsel2 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel2`

Content of register PPG2_PDSEL2.

5.35.2.12 ppg2_pdsel3 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel3`

Content of register PPG2_PDSEL3.

5.35.2.13 ppg2_pdsel4 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel4`

Content of register PPG2_PDSEL4.

5.35.2.14 ppg2_pdsel5 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel5`

Content of register PPG2_PDSEL5.

5.35.2.15 ppg2_pdsel6 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel6`

Content of register PPG2_PDSEL6.

5.35.2.16 ppg2_pdsel7 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel7`

Content of register PPG2_PDSEL7.

5.35.2.17 ppg2_pdsel8 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_pdsel8`

Content of register PPG2_PDSEL8.

5.35.2.18 ppg2_afesel1 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_afesel1`

Content of register PPG2_AFESEL1.

5.35.2.19 ppg2_afesel2 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_afesel2`

Content of register PPG2_AFESEL2.

5.35.2.20 ppg2_afesel3 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_afesel3`

Content of register PPG2_AFESEL3.

5.35.2.21 ppg2_afesel4 `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_afesel4`

Content of register PPG2_AFESEL4.

5.35.2.22 ppg2_afeen `uint8_t as7058_reg_group_pd_t::pd_regs::ppg2_afeen`

Content of register PPG2_AFEEN.

5.36 as7058_reg_group_power_t::power_regs Struct Reference

Data Fields

- uint8_t pwr_on
- uint8_t pwr_iso
- uint8_t clk_cfg
- uint8_t ref_cfg1
- uint8_t ref_cfg2
- uint8_t ref_cfg3
- uint8_t standby_on1
- uint8_t standby_on2
- uint8_t standby_en1
- uint8_t standby_en2
- uint8_t standby_en3
- uint8_t standby_en4
- uint8_t standby_en5
- uint8_t standby_en6
- uint8_t standby_en7
- uint8_t standby_en8
- uint8_t standby_en9
- uint8_t standby_en10
- uint8_t standby_en11
- uint8_t standby_en12
- uint8_t standby_en13
- uint8_t standby_en14

5.36.1 Detailed Description

Contents of the Power register group.

5.36.2 Field Documentation

5.36.2.1 pwr_on `uint8_t as7058_reg_group_power_t::power_regs::pwr_on`

Content of register PWR_ON.

5.36.2.2 pwr_iso `uint8_t as7058_reg_group_power_t::power_regs::pwr_iso`

Content of register PWR_ISO.

5.36.2.3 clk_cfg `uint8_t as7058_reg_group_power_t::power_regs::clk_cfg`

Content of register CLK_CFG.

5.36.2.4 ref_cfg1 `uint8_t as7058_reg_group_power_t::power_regs::ref_cfg1`

Content of register REF_CFG1.

5.36.2.5 ref_cfg2 `uint8_t as7058_reg_group_power_t::power_regs::ref_cfg2`

Content of register REF_CFG2.

5.36.2.6 ref_cfg3 `uint8_t as7058_reg_group_power_t::power_regs::ref_cfg3`

Content of register REF_CFG3.

5.36.2.7 standby_on1 `uint8_t as7058_reg_group_power_t::power_regs::standby_on1`

Content of register STANDBY_ON1.

5.36.2.8 standby_on2 `uint8_t as7058_reg_group_power_t::power_regs::standby_on2`

Content of register STANDBY_ON2.

5.36.2.9 standby_en1 `uint8_t as7058_reg_group_power_t::power_regs::standby_en1`

Content of register STANDBY_EN1.

5.36.2.10 standby_en2 `uint8_t as7058_reg_group_power_t::power_regs::standby_en2`

Content of register STANDBY_EN2.

5.36.2.11 standby_en3 `uint8_t as7058_reg_group_power_t::power_regs::standby_en3`

Content of register STANDBY_EN3.

5.36.2.12 standby_en4 `uint8_t as7058_reg_group_power_t::power_regs::standby_en4`

Content of register STANDBY_EN4.

5.36.2.13 standby_en5 `uint8_t as7058_reg_group_power_t::power_regs::standby_en5`

Content of register STANDBY_EN5.

5.36.2.14 standby_en6 `uint8_t as7058_reg_group_power_t::power_regs::standby_en6`

Content of register STANDBY_EN6.

5.36.2.15 standby_en7 `uint8_t as7058_reg_group_power_t::power_regs::standby_en7`

Content of register STANDBY_EN7.

5.36.2.16 standby_en8 `uint8_t as7058_reg_group_power_t::power_regs::standby_en8`

Content of register STANDBY_EN8.

5.36.2.17 standby_en9 `uint8_t as7058_reg_group_power_t::power_regs::standby_en9`

Content of register STANDBY_EN9.

5.36.2.18 standby_en10 `uint8_t as7058_reg_group_power_t::power_regs::standby_en10`

Content of register STANDBY_EN10.

5.36.2.19 standby_en11 `uint8_t as7058_reg_group_power_t::power_regs::standby_en11`

Content of register STANDBY_EN11.

5.36.2.20 standby_en12 `uint8_t as7058_reg_group_power_t::power_regs::standby_en12`

Content of register STANDBY_EN12.

5.36.2.21 standby_en13 `uint8_t as7058_reg_group_power_t::power_regs::standby_en13`

Content of register STANDBY_EN13.

5.36.2.22 standby_en14 `uint8_t as7058_reg_group_power_t::power_regs::standby_en14`

Content of register STANDBY_EN14.

5.37 as7058_reg_group_pp_t::pp_regs Struct Reference

Data Fields

- uint8_t pp_cfg
- uint8_t ppg1_pp1
- uint8_t ppg1_pp2
- uint8_t ppg2_pp1
- uint8_t ppg2_pp2

5.37.1 Detailed Description

Contents of the Post-Processing register group.

5.37.2 Field Documentation

5.37.2.1 pp_cfg `uint8_t as7058_reg_group_pp_t::pp_regs::pp_cfg`

Content of register PP_CFG.

5.37.2.2 ppg1_pp1 `uint8_t as7058_reg_group_pp_t::pp_regs::ppg1_pp1`

Content of register PPG1_PP1.

5.37.2.3 ppg1_pp2 `uint8_t as7058_reg_group_pp_t::pp_regs::ppg1_pp2`

Content of register PPG1_PP2.

5.37.2.4 ppg2_pp1 `uint8_t as7058_reg_group_pp_t::pp_regs::ppg2_pp1`

Content of register PPG2_PP1.

5.37.2.5 ppg2_pp2 `uint8_t as7058_reg_group_pp_t::pp_regs::ppg2_pp2`

Content of register PPG2_PP2.

5.38 as7058_reg_group_ppg_t::ppg_regs Struct Reference

Data Fields

- uint8_t ppgmod_cfg1
- uint8_t ppgmod_cfg2
- uint8_t ppgmod_cfg3
- uint8_t ppgmod1_cfg1
- uint8_t ppgmod1_cfg2
- uint8_t ppgmod1_cfg3
- uint8_t ppgmod2_cfg1
- uint8_t ppgmod2_cfg2
- uint8_t ppgmod2_cfg3

5.38.1 Detailed Description

Contents of the PPG register group.

5.38.2 Field Documentation

5.38.2.1 ppgmod_cfg1 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod_cfg1`

Content of register PPGMOD_CFG1.

5.38.2.2 ppgmod_cfg2 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod_cfg2`

Content of register PPGMOD_CFG2.

5.38.2.3 ppgmod_cfg3 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod_cfg3`

Content of register PPGMOD_CFG3.

5.38.2.4 ppgmod1_cfg1 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod1_cfg1`

Content of register PPGMOD1_CFG1.

5.38.2.5 ppgmod1_cfg2 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod1_cfg2`

Content of register PPGMOD1_CFG2.

5.38.2.6 ppgmod1_cfg3 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod1_cfg3`

Content of register PPGMOD1_CFG3.

5.38.2.7 ppgmod2_cfg1 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod2_cfg1`

Content of register PPGMOD2_CFG1.

5.38.2.8 ppgmod2_cfg2 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod2_cfg2`

Content of register PPGMOD2_CFG2.

5.38.2.9 ppgmod2_cfg3 `uint8_t as7058_reg_group_ppg_t::ppg_regs::ppgmod2_cfg3`

Content of register PPGMOD2_CFG3.

5.39 as7058_reg_group_seq_t::seq_regs Struct Reference

Data Fields

- uint8_t irq_enable
- uint8_t ppg_sub_wait
- uint8_t ppg_sar_wait
- uint8_t ppg_led_init
- uint8_t ppg_freql
- uint8_t ppg_freqh
- uint8_t ppg1_sub_en
- uint8_t ppg2_sub_en
- uint8_t ppg_mode_1
- uint8_t ppg_mode_2
- uint8_t ppg_mode_3
- uint8_t ppg_mode_4
- uint8_t ppg_mode_5
- uint8_t ppg_mode_6
- uint8_t ppg_mode_7
- uint8_t ppg_mode_8
- uint8_t ppg_cfg
- uint8_t ecg_freql
- uint8_t ecg_freqh
- uint8_t ecg1_freqdivl
- uint8_t ecg1_freqdivh
- uint8_t ecg2_freqdivl
- uint8_t ecg2_freqdivh
- uint8_t ecg_subs
- uint8_t leadoff_initl
- uint8_t leadoff_inith
- uint8_t ecg_initl
- uint8_t ecg_inith
- uint8_t sample_num

5.39.1 Detailed Description

Contents of the Sequencer register group.

5.39.2 Field Documentation

5.39.2.1 irq_enable `uint8_t as7058_reg_group_seq_t::seq_regs::irq_enable`

Content of register IRQ_ENABLE.

5.39.2.2 ppg_sub_wait `uint8_t as7058_reg_group_seq_t::seq_regs::ppg_sub_wait`

Content of register PPG_SUB_WAIT.

5.39.2.3 ppg_sar_wait `uint8_t as7058_reg_group_seq_t::seq_regs::ppg_sar_wait`

Content of register PPG_SAR_WAIT.

5.39.2.4 ppg_led_init `uint8_t as7058_reg_group_seq_t::seq_regs::ppg_led_init`

Content of register PPG_LED_INIT.

5.39.2.5 ppg_freql `uint8_t as7058_reg_group_seq_t::seq_regs::ppg_freql`

Content of register PPG_FREQL.

5.39.2.6 ppg_freqh `uint8_t as7058_reg_group_seq_t::seq_regs::ppg_freqh`

Content of register PPG_FREQH.

5.39.2.7 ppg1_sub_en `uint8_t as7058_reg_group_seq_t::seq_regs::ppg1_sub_en`

Content of register PPG1_SUB_EN.

5.39.2.8 ppg2_sub_en `uint8_t as7058_reg_group_seq_t::seq_regs::ppg2_sub_en`

Content of register PPG2_SUB_EN.

5.39.2.9 ppg_mode_1 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_1

Content of register PPG_MODE1.

5.39.2.10 ppg_mode_2 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_2

Content of register PPG_MODE2.

5.39.2.11 ppg_mode_3 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_3

Content of register PPG_MODE3.

5.39.2.12 ppg_mode_4 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_4

Content of register PPG_MODE4.

5.39.2.13 ppg_mode_5 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_5

Content of register PPG_MODE5.

5.39.2.14 ppg_mode_6 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_6

Content of register PPG_MODE6.

5.39.2.15 ppg_mode_7 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_7

Content of register PPG_MODE7.

5.39.2.16 ppg_mode_8 uint8_t as7058_reg_group_seq_t::seq_regs::ppg_mode_8

Content of register PPG_MODE8.

5.39.2.17 ppg_cfg uint8_t as7058_reg_group_seq_t::seq_regs::ppg_cfg

Content of register PPG_CFG.

5.39.2.18 ecg_freql uint8_t as7058_reg_group_seq_t::seq_regs::ecg_freql

Content of register ECG_FREQL.

5.39.2.19 ecg_freqh uint8_t as7058_reg_group_seq_t::seq_regs::ecg_freqh

Content of register ECG_FREQH.

5.39.2.20 ecg1_freqdivl `uint8_t as7058_reg_group_seq_t::seq_regs::ecg1_freqdivl`

Content of register ECG1_FREQDIVL.

5.39.2.21 ecg1_freqdivh `uint8_t as7058_reg_group_seq_t::seq_regs::ecg1_freqdivh`

Content of register ECG1_FREQDIVH.

5.39.2.22 ecg2_freqdivl `uint8_t as7058_reg_group_seq_t::seq_regs::ecg2_freqdivl`

Content of register ECG2_FREQDIVL.

5.39.2.23 ecg2_freqdivh `uint8_t as7058_reg_group_seq_t::seq_regs::ecg2_freqdivh`

Content of register ECG2_FREQDIVH.

5.39.2.24 ecg_subs `uint8_t as7058_reg_group_seq_t::seq_regs::ecg_subs`

Content of register ECG_SUBS.

5.39.2.25 leadoff_initl `uint8_t as7058_reg_group_seq_t::seq_regs::leadoff_initl`

Content of register LEADOFF_INITL.

5.39.2.26 leadoff_inith `uint8_t as7058_reg_group_seq_t::seq_regs::leadoff_inith`

Content of register LEADOFF_INITH.

5.39.2.27 ecg_initl `uint8_t as7058_reg_group_seq_t::seq_regs::ecg_initl`

Content of register ECG_INITL.

5.39.2.28 ecg_inith `uint8_t as7058_reg_group_seq_t::seq_regs::ecg_inith`

Content of register ECG_INITH.

5.39.2.29 sample_num `uint8_t as7058_reg_group_seq_t::seq_regs::sample_num`

Content of register SAMPLE_NUM.

5.40 as7058_reg_group_sinc_t::sinc_regs Struct Reference

Data Fields

- uint8_t ppg_sinc_cfga
- uint8_t ppg_sinc_cfgb
- uint8_t ppg_sinc_cfgc
- uint8_t ppg_sinc_cfgd
- uint8_t ecg1_sinc_cfga
- uint8_t ecg1_sinc_cfgb
- uint8_t ecg1_sinc_cfgc
- uint8_t ecg2_sinc_cfga
- uint8_t ecg2_sinc_cfgb
- uint8_t ecg2_sinc_cfgc
- uint8_t ecg_sinc_cfg

5.40.1 Detailed Description

Contents of the Sinc Filter register group.

5.40.2 Field Documentation

5.40.2.1 ppg_sinc_cfga `uint8_t as7058_reg_group_sinc_t::sinc_regs::ppg_sinc_cfga`

Content of register PPG_SINC_CFGA.

5.40.2.2 ppg_sinc_cfgb `uint8_t as7058_reg_group_sinc_t::sinc_regs::ppg_sinc_cfgb`

Content of register PPG_SINC_CFGB.

5.40.2.3 ppg_sinc_cfgc `uint8_t as7058_reg_group_sinc_t::sinc_regs::ppg_sinc_cfgc`

Content of register PPG_SINC_CFGC.

5.40.2.4 ppg_sinc_cfgd `uint8_t as7058_reg_group_sinc_t::sinc_regs::ppg_sinc_cfgd`

Content of register PPG_SINC_CFGD.

5.40.2.5 ecg1_sinc_cfga `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg1_sinc_cfga`

Content of register ECG1_SINC_CFGA.

5.40.2.6 ecg1_sinc_cfgb `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg1_sinc_cfgb`

Content of register ECG1_SINC_CFGB.

5.40.2.7 ecg1_sinc_cfgc `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg1_sinc_cfgc`

Content of register ECG1_SINC_CFGC.

5.40.2.8 ecg2_sinc_cfga `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg2_sinc_cfga`

Content of register ECG2_SINC_CFGA.

5.40.2.9 ecg2_sinc_cfgb `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg2_sinc_cfgb`

Content of register ECG2_SINC_CFGB.

5.40.2.10 ecg2_sinc_cfgc `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg2_sinc_cfgc`

Content of register ECG2_SINC_CFGC.

5.40.2.11 ecg_sinc_cfg `uint8_t as7058_reg_group_sinc_t::sinc_regs::ecg_sinc_cfg`

Content of register ECG_SINC_CFG.

5.41 status_regs Struct Reference

Data Fields

- uint8_t status_seq
- uint8_t status_led
- uint8_t status_asata
- uint8_t status_asatb
- uint8_t status_vcsel
- uint8_t status_vcsel_vss
- uint8_t status_vcsel_vdd
- uint8_t status_leadoff
- uint8_t status_iir

5.41.1 Detailed Description

Register group for sensor status events.

5.41.2 Field Documentation

5.41.2.1 status_seq `uint8_t status_regs::status_seq`

Content of register STATUS_SEQ.

5.41.2.2 status_led `uint8_t status_regs::status_led`

Content of register STATUS_LED.

5.41.2.3 status_asata `uint8_t status_regs::status_asata`

Content of register STATUS_ASATA.

5.41.2.4 status_asatb `uint8_t status_regs::status_asatb`

Content of register STATUS_ASATB.

5.41.2.5 status_vcsel `uint8_t status_regs::status_vcsel`

Content of register STATUS_VCSEL.

5.41.2.6 status_vcsel_vss `uint8_t status_regs::status_vcsel_vss`

Content of register STATUS_VCSEL_VSS.

5.41.2.7 status_vcsel_vdd `uint8_t status_regs::status_vcsel_vdd`

Content of register STATUS_VCSEL_VDD.

5.41.2.8 status_leadoff `uint8_t status_regs::status_leadoff`

Content of register STATUS_LEADOFF.

5.41.2.9 status_iir `uint8_t status_regs::status_iir`

TRUE when IIR interrupt set, otherwise FALSE.